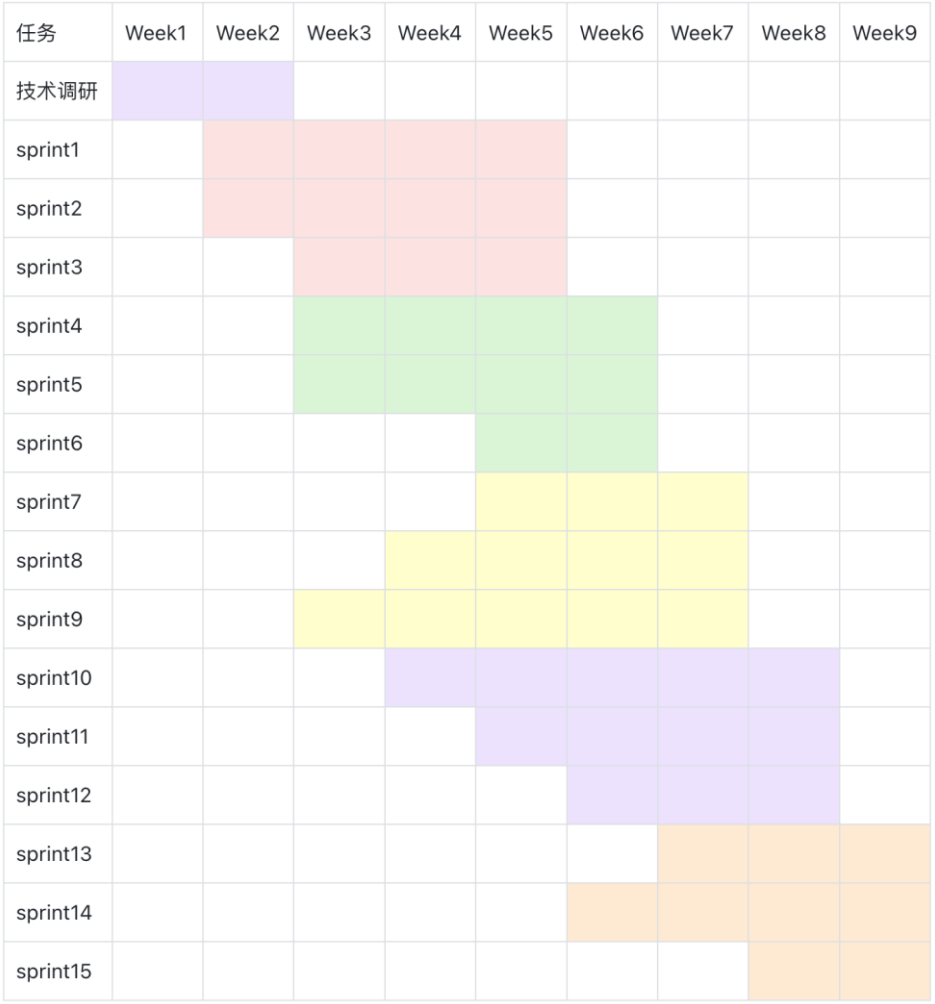
北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

卓越工程
第一组终期汇报

汇报人：

时　间：2026/10/22

工作日志



我们的项目一共完成了15个Sprint：

Sprint 1: 项目初始化与核心架构搭建
Sprint 2: RAG 检索系统实现
Sprint 3: 多语言支持与国际化
Sprint 4: AI Agent 智能助教系统
Sprint 5: 资源管理与转换系统
Sprint 6: 知识图谱可视化系统
Sprint 7: 总结模块
Sprint 8: 学习计划模块
Sprint 9: FastAPI 及前端模块
Sprint 10: 大模型微调系统
Sprint 11: 题目模块
Sprint 12: 管理员模块
Sprint 13: 教师模块
Sprint 14: EasyOCR 模块
Sprint 15: 项目整体优化

章节内容

I. 功能覆盖对比

功能覆盖

用例图 (教师、学生、管理员)

II. 人机协同(HI+AI)

工作流程、分工表、并行工作流、学习闭环、数据流

项目背景与目标

背景阐述:

- 传统教育模式存在个性化不足、资源分配不均、学习效果难以量化等痛点
- 大数据与人工智能技术为智能教育提供了新的解决方案
- 在线学习平台数据的爆发式增长为精准教学提供了数据基础

核心目标:

- 构建一个融合 HI(Human Intelligence) 与 AI 的智能化学习辅助系统
- 实现知识的结构化组织、智能化推送和个性化学习路径规划
- 通过多 Agent 协同提供全方位学习支持(知识总结、测验生成、学习计划)

项目概览

项目功能完成情况总览

📊 功能点完成情况

✅ 基础架构	100%
✅ 知识图谱系统	100%
✅ RAG智能问答	100%
✅ 个性化推荐	100%
✅ AI内容生成	95%
✅ 测试评估系统	100%
✅ 多Agent架构	100%
✅ 前端交互系统	100%
✅ 数据管理系统	100%

🎯 项目总体完成度: 99%

- ✨ 核心功能：9/9 模块 100% 完成
- 🚀 系统状态：可投入实际教学使用
- 📈 测试验证：通过完整功能测试

本项目规划了9大核心功能模块，
目前已**全部完成并通过测试。**
系统整体完成度达到**99%**，
已具备**投入实际教学使用**的条件。

项目概览：功能覆盖

功能列表

1. 核心基础架构 ✅ 100%

1. ✅ **前后端分离架构**：采用 FastAPI + React 技术栈，实现高效通信
2. ✅ **多 Agent 协作框架**：完整的 Agent 通信与协调机制
3. ✅ **数据库系统**：完整的关系型数据库设计与实现
4. ✅ **日志追踪系统**：完整的 LLM 调用日志记录与分析
5. ✅ **多语言支持**：智能语言检测，支持中英文无缝切换
6. ✅ **EasyOCR**：OCR 提取文字功能，支持从图片等提取文字

亮点：系统架构清晰，可扩展性强，为后续功能提供坚实基础

2. 知识图谱系统 ✅ 100%

1. ✅ **三层知识图谱**：完整构建 children-grandchildren-great-grandchildren 层级结构
2. ✅ **知识关系网络**：实现知识点间的关联与依赖关系
3. ✅ **学习进度追踪**：基于 flag 的智能进度管理系统
4. ✅ **递归更新机制**：子节点完成自动更新父节点状态
5. ✅ **资源关联**：PDF 文档与知识点精准映射
6. ✅ **可视化展示**：树形结构直观展示知识体系

亮点：知识图谱涵盖大数据分析全部知识点，支持个性化学习路径

项目概览：功能覆盖

功能列表

3. RAG智能问答系统 ✅ 100%

1. ✅ **向量数据库**：基于 Chroma 的高效向量存储与检索
2. ✅ **智能去重**：基于内容哈希的文档去重机制
3. ✅ **多文档检索**：支持跨文档的语义检索
4. ✅ **上下文增强**：检索结果与 LLM 生成深度融合
5. ✅ **多格式支持**：支持 PDF、DOCX 等多种文档格式
6. ✅ **精准检索**：针对不同场景的定制化检索策略
7. ✅ **容错机制**：检索失败时自动切换到直接 LLM 生成

亮点：RAG 系统响应速度快，答案准确率高，已通过大量测试验证

4. 个性化学习推荐系统 ✅ 100%

1. ✅ **智能诊断**：基于测验结果的知识薄弱点精准识别
2. ✅ **三级优先级**：High/Medium/Low 优先级智能分配
3. ✅ **学习计划生成**：自动生成个性化时间线学习计划
4. ✅ **材料推荐**：基于知识点的学习资源智能推荐
5. ✅ **数据持久化**：学习计划自动保存，支持历史查看
6. ✅ **推荐算法**：结合 RAG 检索的资源推荐机制

亮点：推荐系统能够根据学生特点生成高度个性化的学习方案

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

功能列表

5. AI内容生成与质量控制 ✅ 95%

1. ✅ **知识总结生成**：自动生成章节知识总结与要点提炼
2. ✅ **题目自动生成**：高质量测验题目智能生成
3. ✅ **质量控制**：严格的提示词工程保证题目专业性
4. ✅ **格式规范**：强制 JSON 输出，保证数据结构一致性
5. ✅ **多语言生成**：支持中英文内容自适应生成
6. ✅ **迷惑项设计**：选择题干扰项具有高度迷惑性
7. ⚠ AI生成内容的自动化质量评分
8. ⚠ 内容安全验证机制完善

亮点：AI生成内容质量经过多轮测试优化，达到教学使用标准

6. 测试评估系统 ✅ 100%

1. ✅ **测验生成**：基于知识图谱的智能测验生成
2. ✅ **主题提取**：自动从知识图谱提取可测试主题
3. ✅ **结果分析**：自动识别弱项/中等/优势领域
4. ✅ **数据可视化**：测验结果柱状图展示
5. ✅ **学习闭环**：测验结果直接驱动学习计划生成

•**亮点**：测试系统完整覆盖评估、反馈、推荐全流程

项目概览：功能覆盖

功能列表

7. 多Agent协作架构 ✅ 100%

1. ✅ **Coordiantor Agent**：负责意图识别与任务分解
2. ✅ **Plan Agent**：负责学习计划生成与优化
3. ✅ **Quiz Agent**：负责测验题目生成、主题提取与答案评分
4. ✅ **Summary Agent**：负责知识总结生成
5. ✅ **QA Agent**：负责 AI 助教的问答
6. ✅ **Agent 通信**：完整的 Agent 间消息传递与协调机制

亮点：多Agent协作使系统功能解耦，易于维护和扩展

8. 前端交互系统 ✅ 100%

1. ✅ **知识图谱可视化**：树形结构动态展示
2. ✅ **学习进度仪表盘**：章节、小节完成度实时显示
3. ✅ **测验交互界面**：流畅的答题体验
4. ✅ **PDF在线阅读**：内嵌PDF浏览器
5. ✅ **学习计划展示**：时间线形式展示学习安排
6. ✅ **系统日志查看**：LLM 调用记录可视化
7. ✅ **响应式设计**：适配多种屏幕尺寸

亮点：界面简洁美观，用户体验流畅，已通过可用性测试

项目概览：功能覆盖

功能列表

9. 数据管理系统 ✅ **100%**

1. ✅ **用户管理**：学生、教师、管理员账户体系
2. ✅ **学习数据**：学习计划、进度、测验记录完整存储
3. ✅ **向量数据库**：支持大规模文档向量化存储
4. ✅ **课程资源**：PDF 文档的组织与管理
5. ✅ **系统日志**：完整的操作日志与性能监控

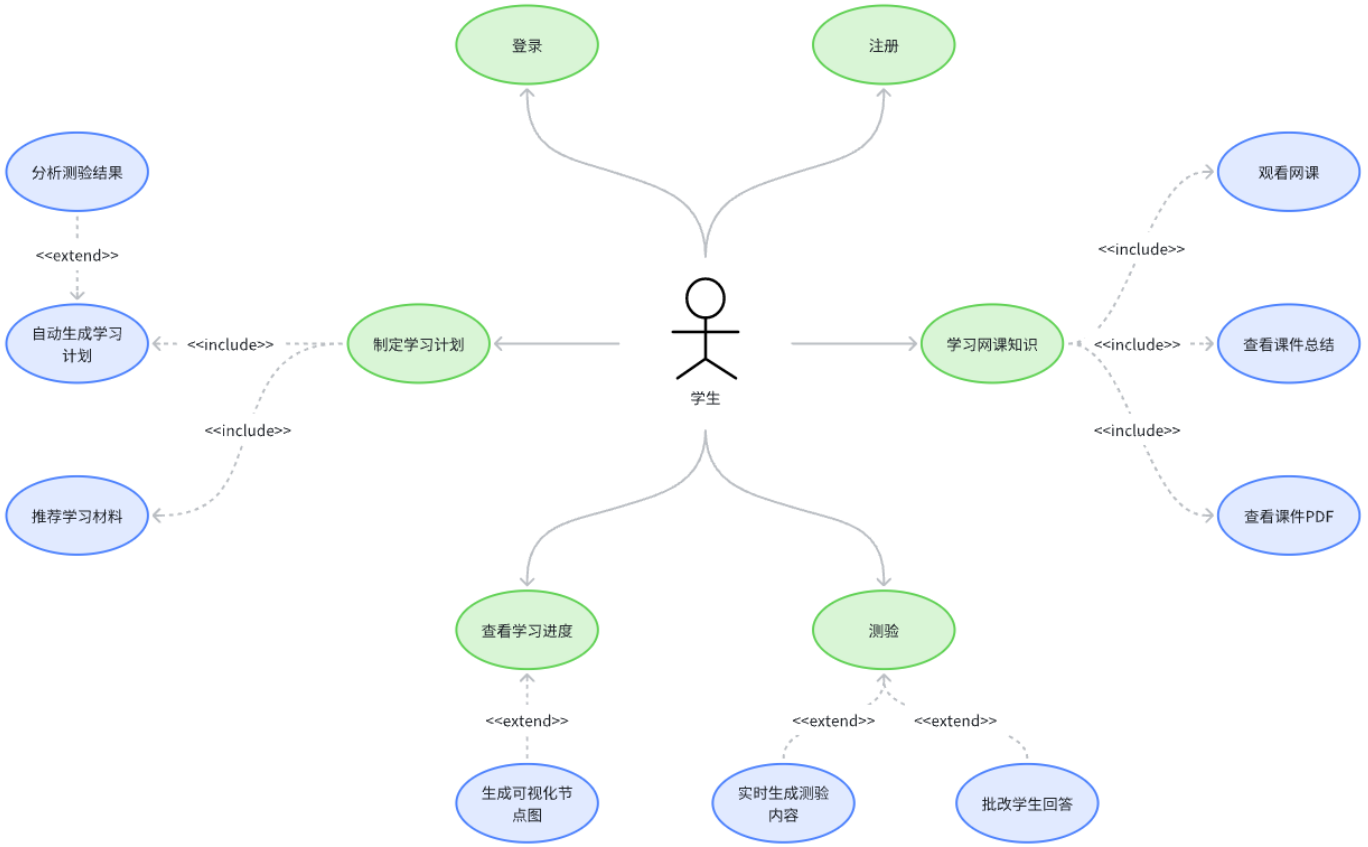
亮点：数据管理规范，支持数据分析与系统优化

总的来说

在我们预设的56个主要功能点中，
我们非常好的完成了其中54个。

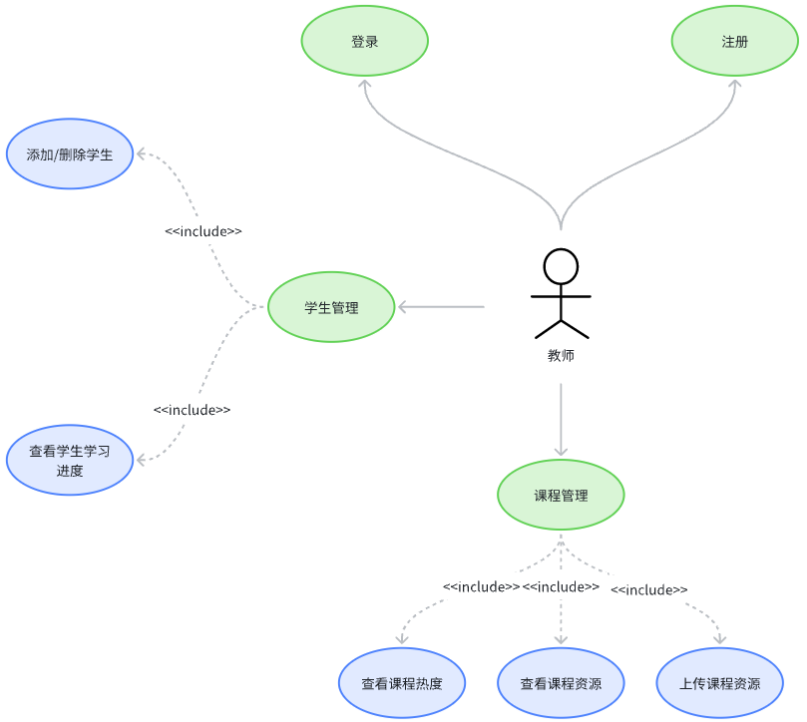


项目概览：功能覆盖

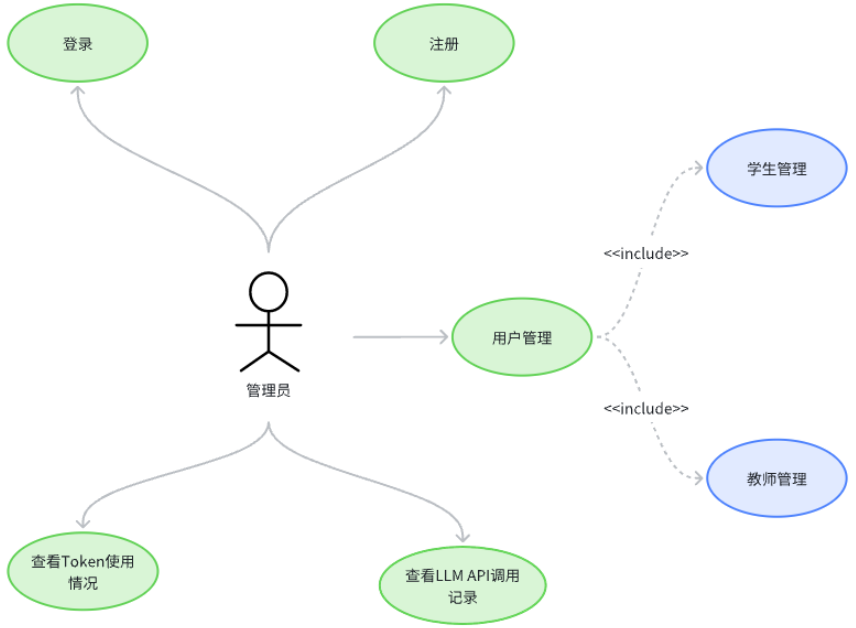


用例图 学生

项目概览：功能覆盖

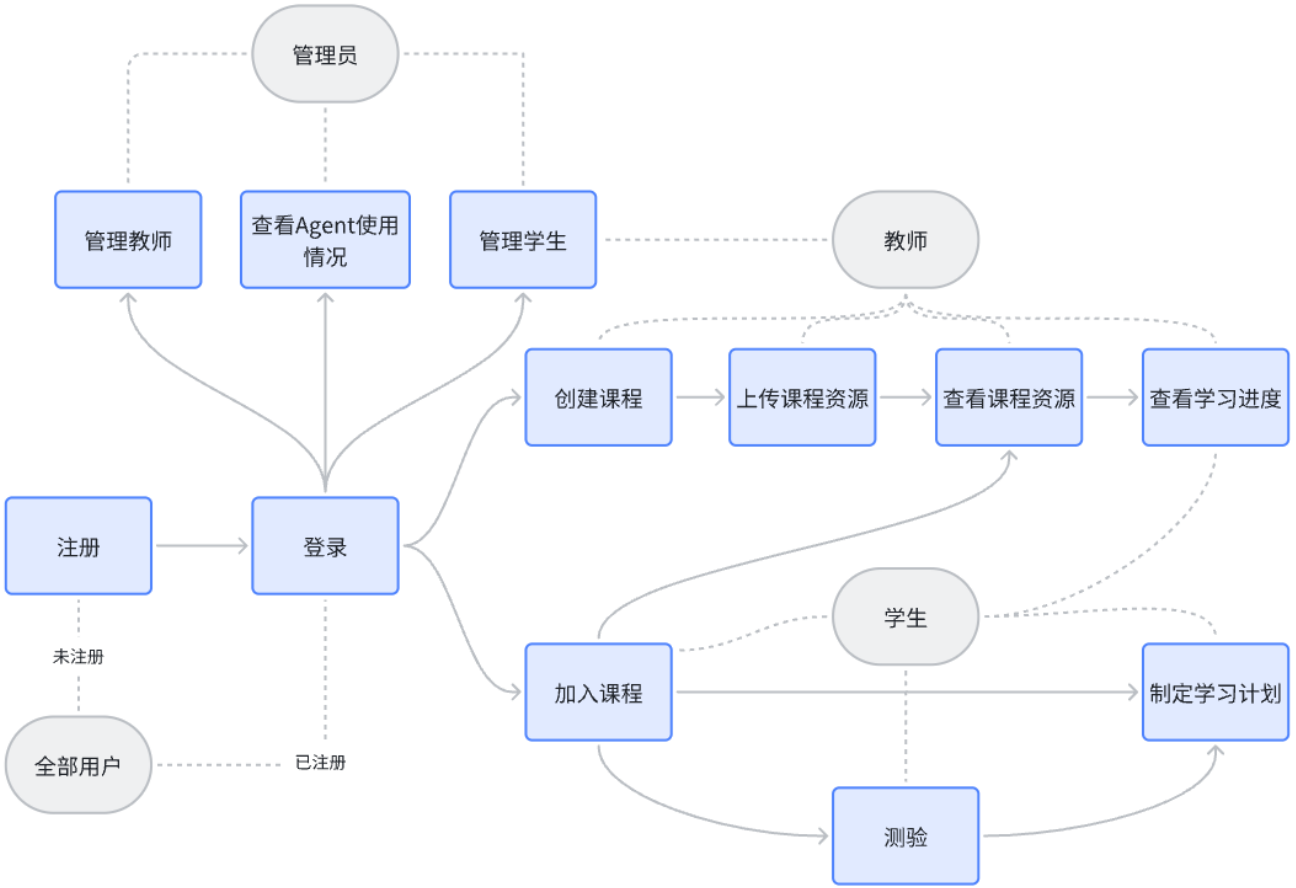


用例图 教师



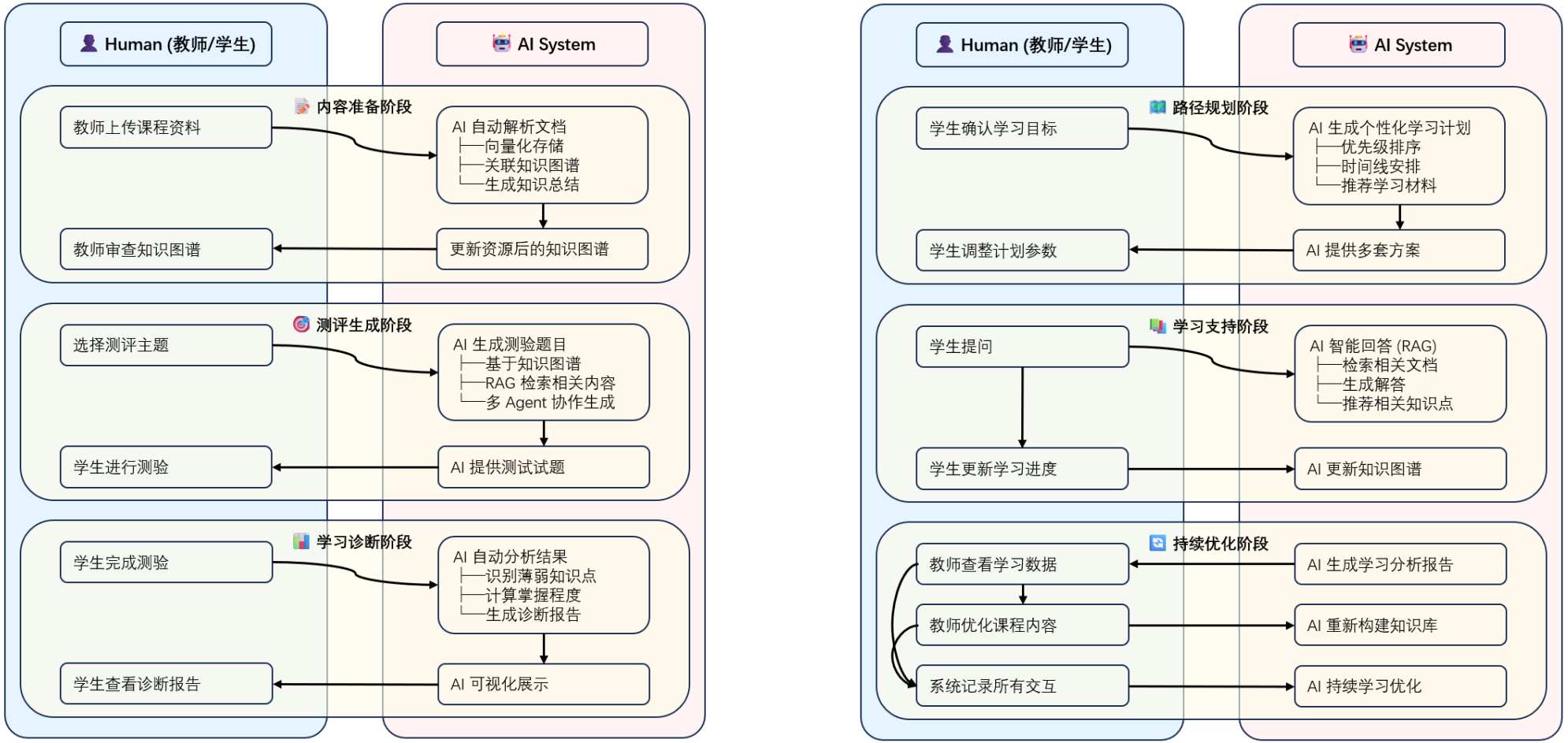
用例图 管理员

项目概览：功能覆盖



项目总体流程图

项目概览：人机协同

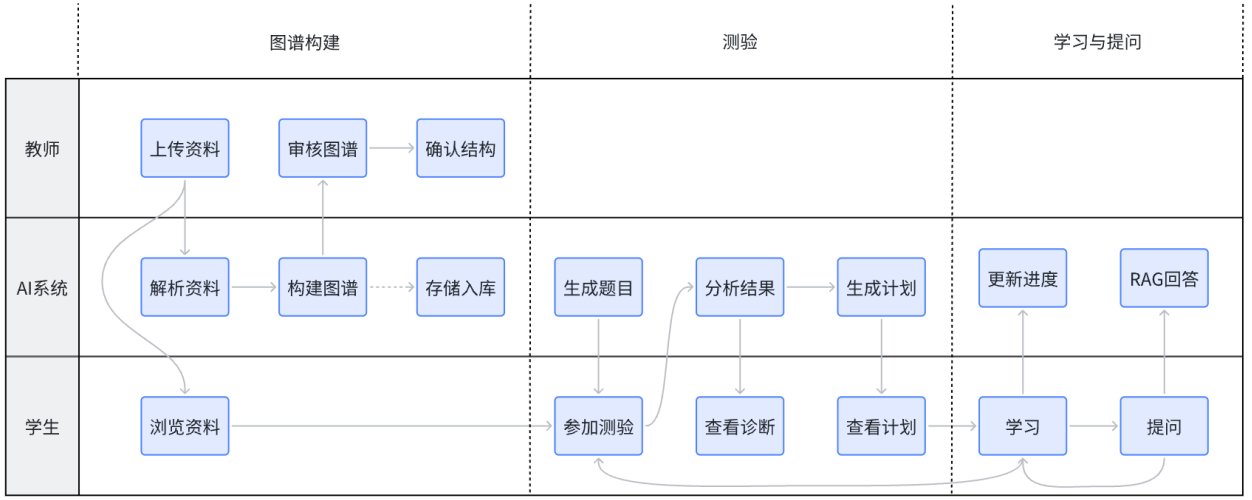


人机协同 (HI+AI) 工作流程

1.2 项目概览：人机协同

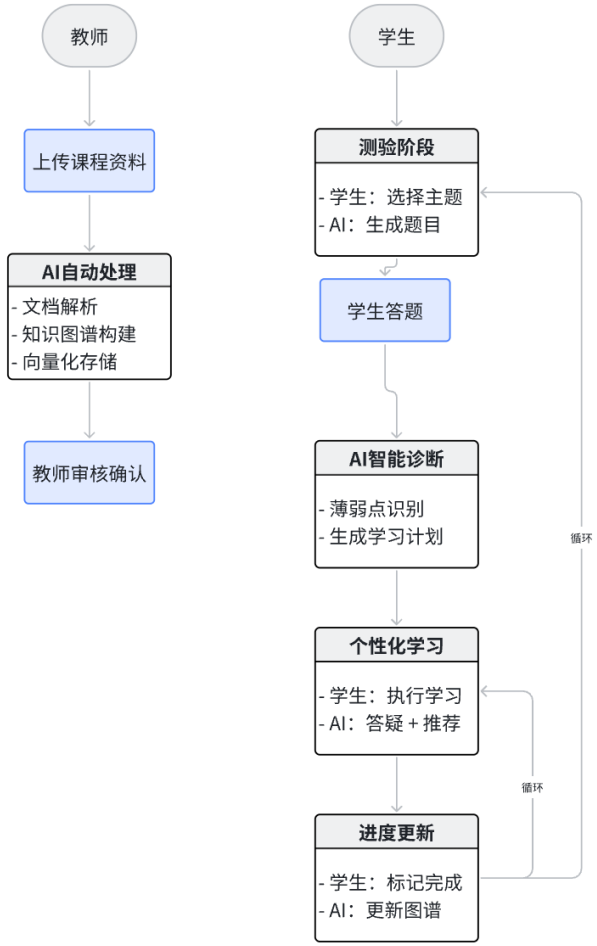
工作环节	Human负责	AI负责
资源准备	上传课程资料 审核知识体系 确认课程结构	自动文档结项 向量化存储 知识图谱资源关联
内容生成	选择测评主题 审核题目质量 调整难度分布	自动生成题目 生成知识总结 推荐学习资料
学习诊断	完成测验答题 查看诊断报告 确认学习目标	自动批改平分 薄弱点识别 生成诊断可视化
路径规划	设定学习偏好 调整学习节奏 选择学习资源	生成个性化推荐 优先级智能排序 多维度资源推荐
互动答疑	提出问题 标记难点 反馈答案质量	RAG 智能检索 上下文理解生成 关联知识推荐
进度跟踪	完成资源学习 查看学习统计 选择学习方向	自动更新知识图谱 生成可视化进度 推荐下一步行动
系统优化	审核 AI 输出质量 提供反馈数据 优化课程设计	记录交互日志 持续学习优化 生成分析报告

人机协同分工表

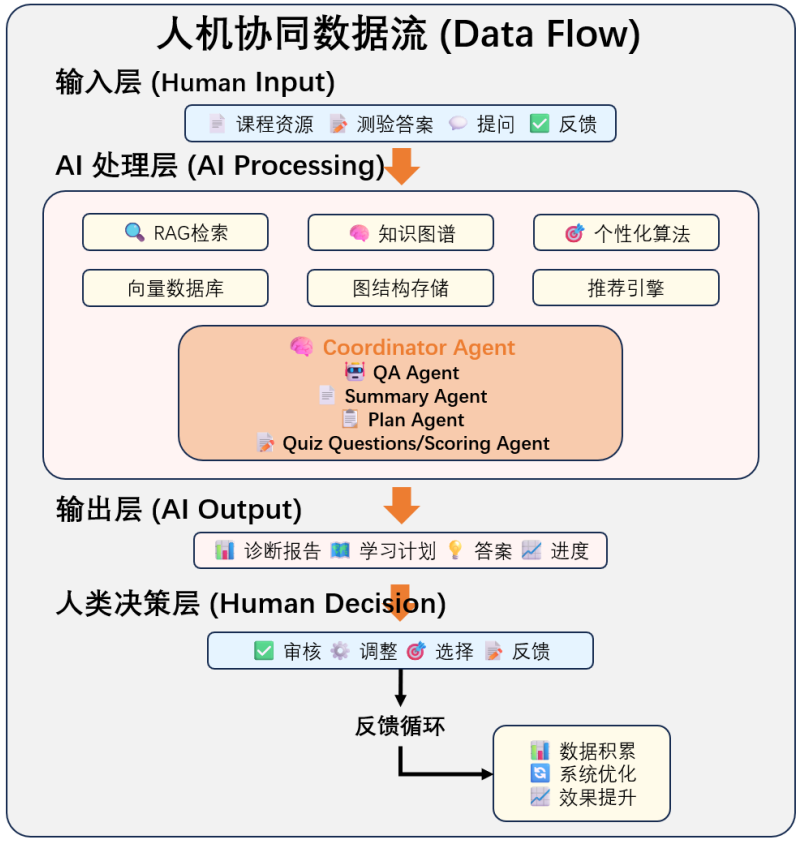


人机协同并行工作流

1.2 项目概览：人机协同



人机协同学习闭环



协同原则:

- AI处理重复性、计算密集型任务
- Human负责决策、审核、创造性工作
- 形成"Human in the Loop"完整闭环

章节内容

I. 业务流程

- 业务整体流程
- 业务核心循环流程
- 业务流程与数据流

II. 系统整体框架

- 功能架构图
- 技术架构图

III. 多Agent架构设计图

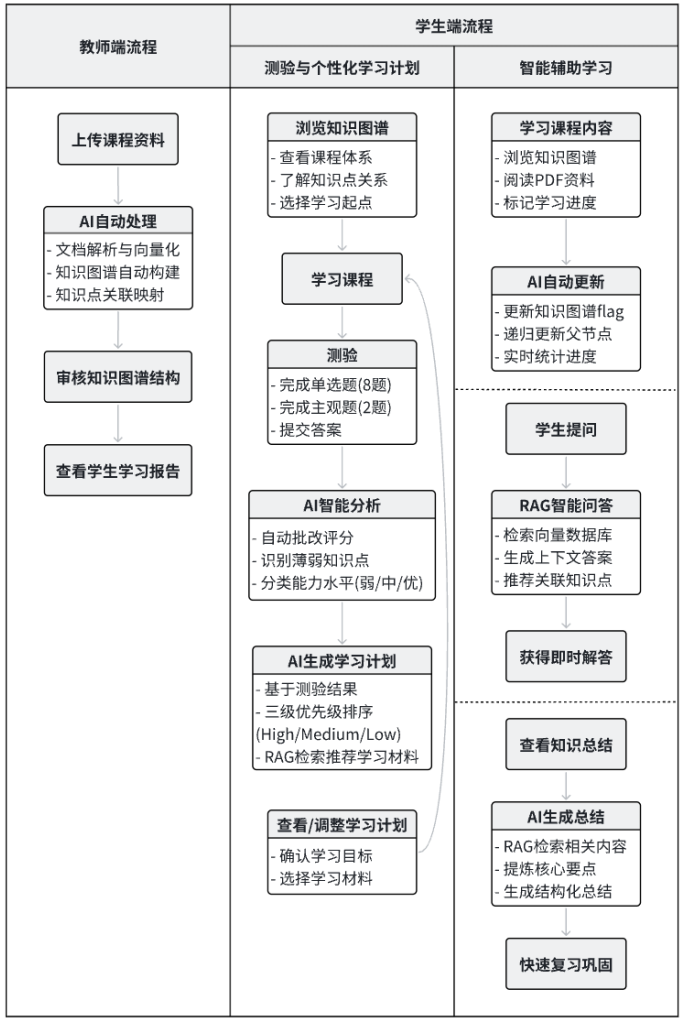
IV. 前后端集成架构

- 整体架构层次
- 数据流向图

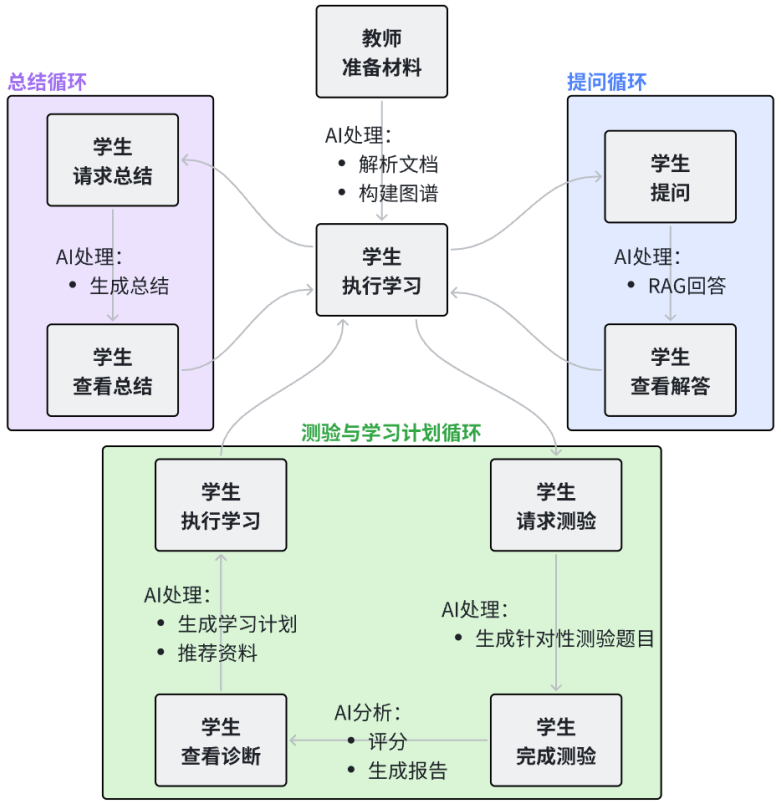
V. 数据层设计

- 整体ER图
- 主要数据表结构
- 数据库类图
- 数据流转示意图

2.1 系统架构设计：业务流程



业务整体流程



业务核心循环流程

系统架构设计：业务流程

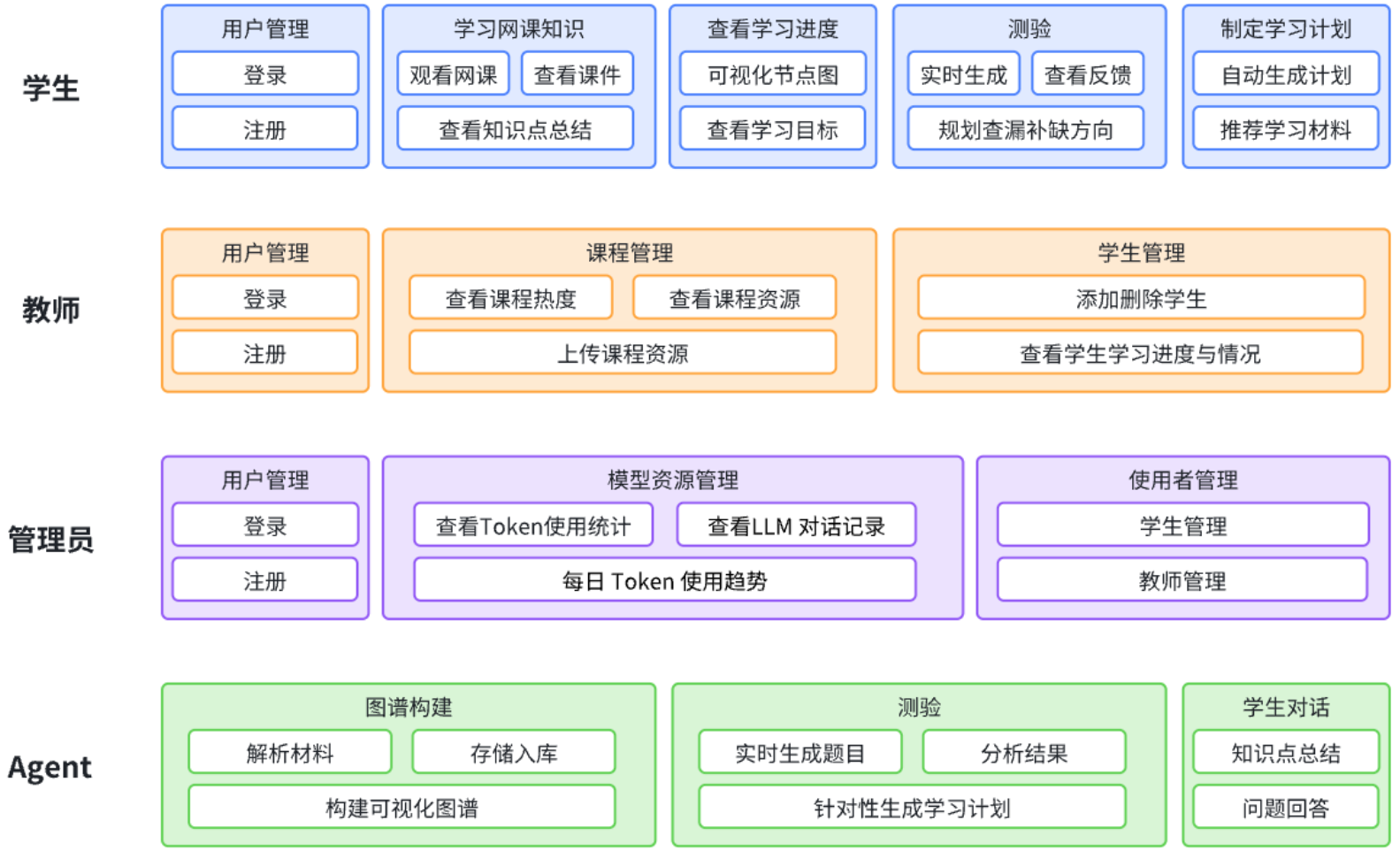


业务流程与数据流

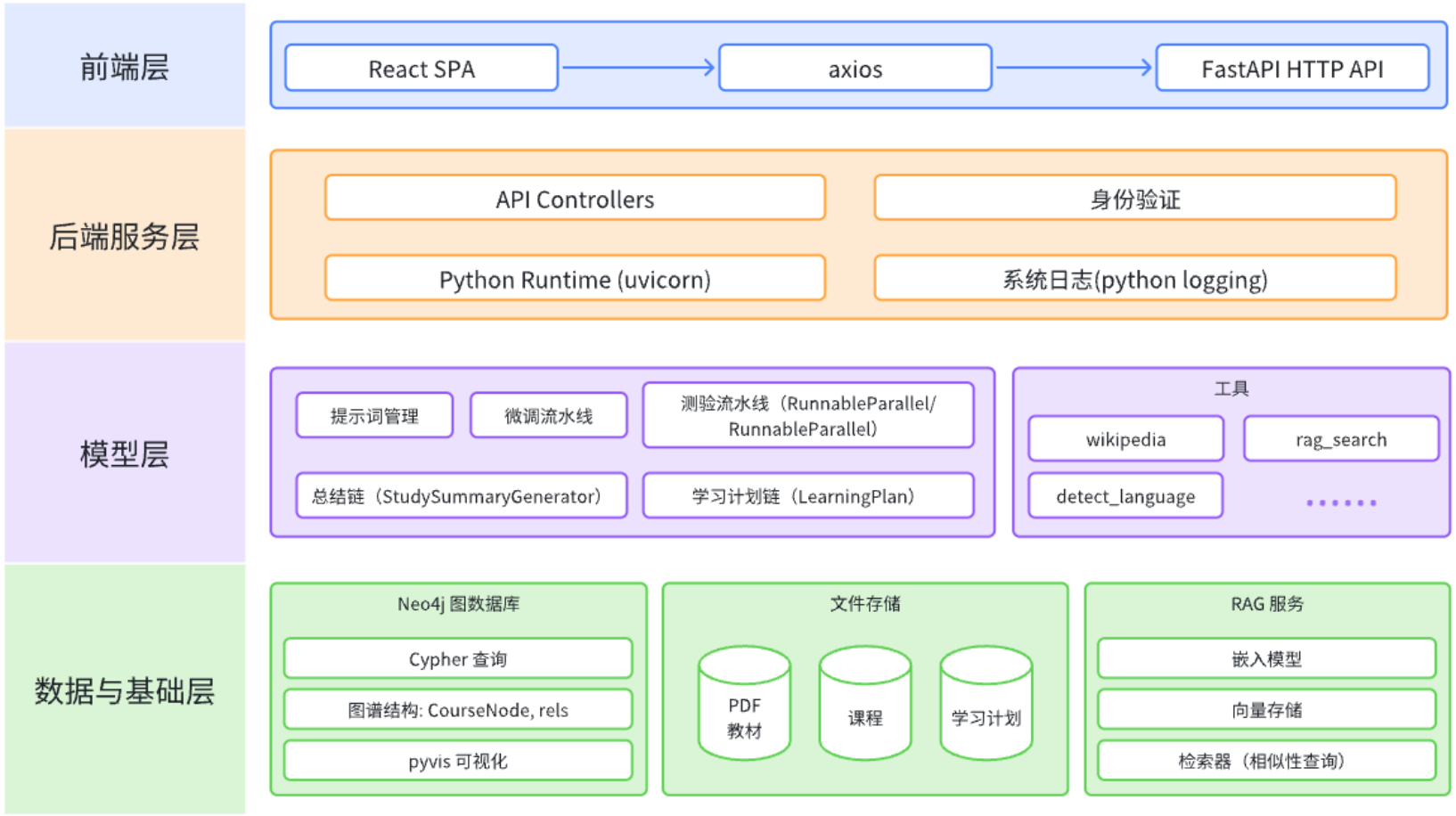
业务流程重点：

1.双端协同：教师准备内容,学生执行学习
2.四阶段闭环：评估→规划→学习→反馈
3.AI 全程辅助：每个环节都有 AI 智能支持
4.数据驱动：基于数据分析持续优化

系统架构设计：系统整体框架

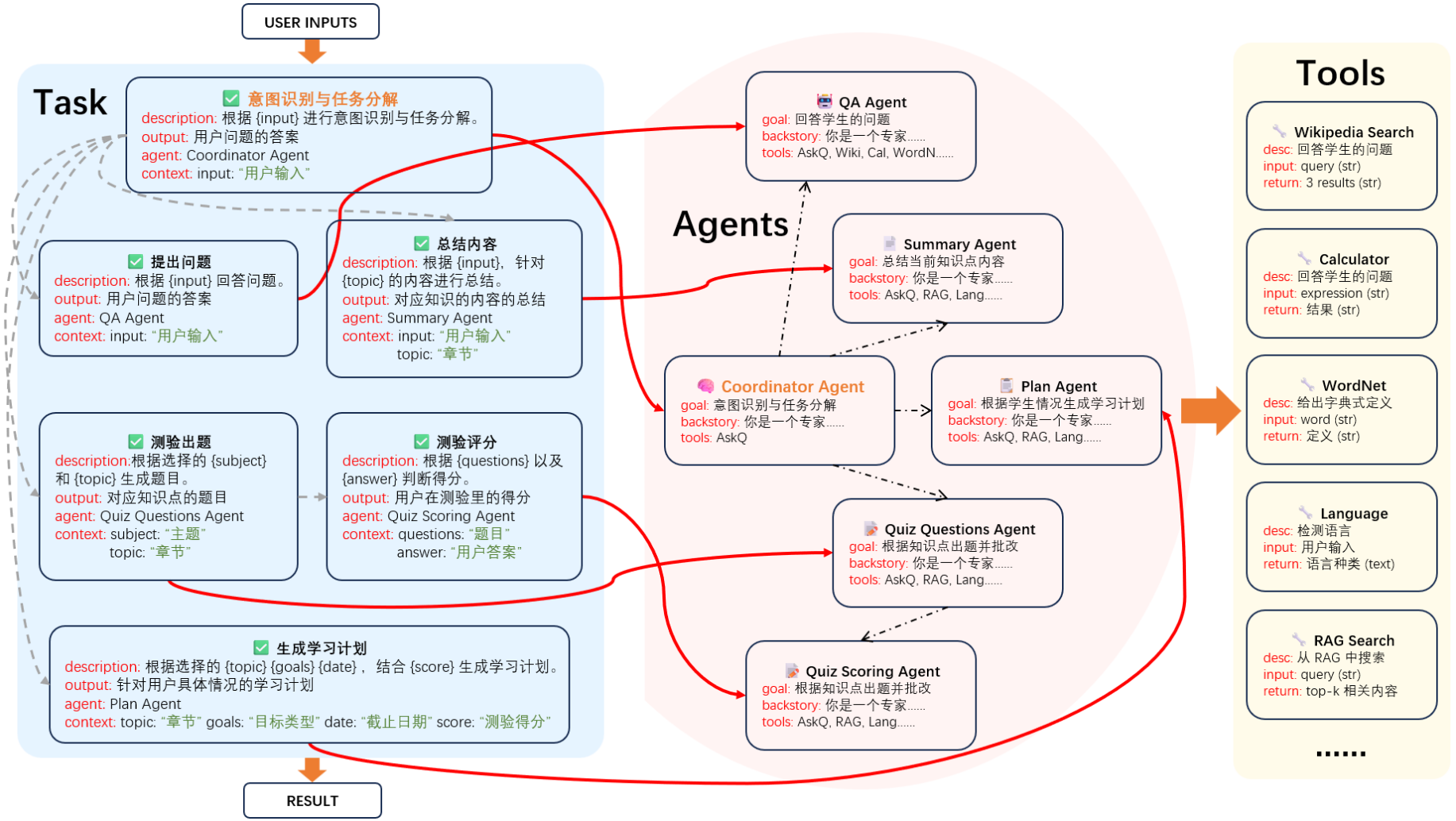


功能架构图


前端层
React SPA
axios
FastAPI HTTP API
后端服务层
API Controllers
身份验证
Python Runtime (uvicorn)
系统日志(python logging)
模型层
提示词管理
微调流水线
测验流水线（RunnableParallel/ RunnableParallel）
总结链（StudySummaryGenerator）
学习计划链（LearningPlan）
工具
wikipedia
rag_search
detect_language
.....
数据与基础层
Neo4j 图数据库
Cypher 查询
图谱结构: CourseNode, rels
pyvis 可视化
文件存储
PDF 教材
课程
学习计划
RAG 服务
嵌入模型
向量存储
检索器（相似性查询）

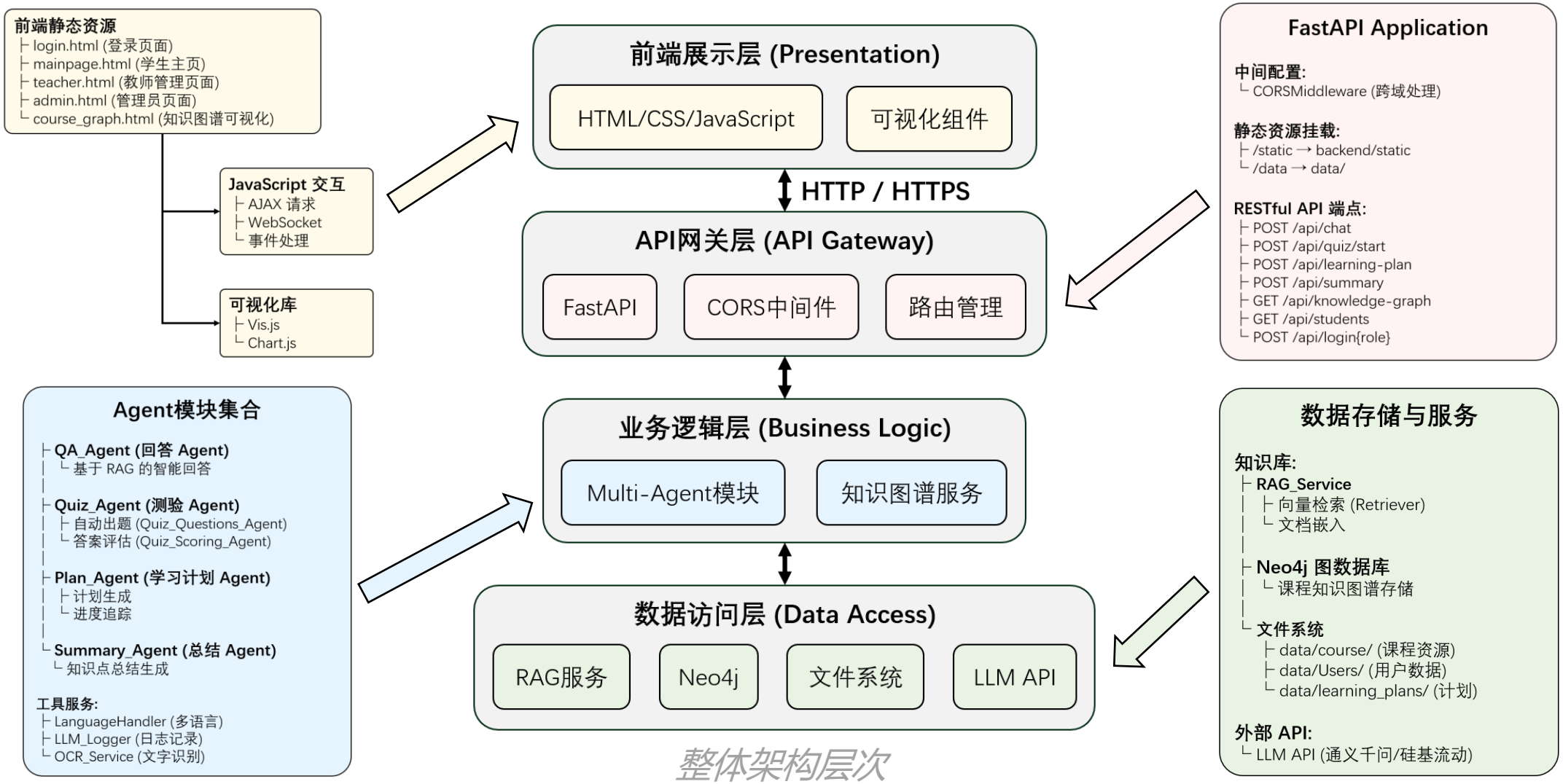
技术架构图

系统架构设计：多Agent架构设计图



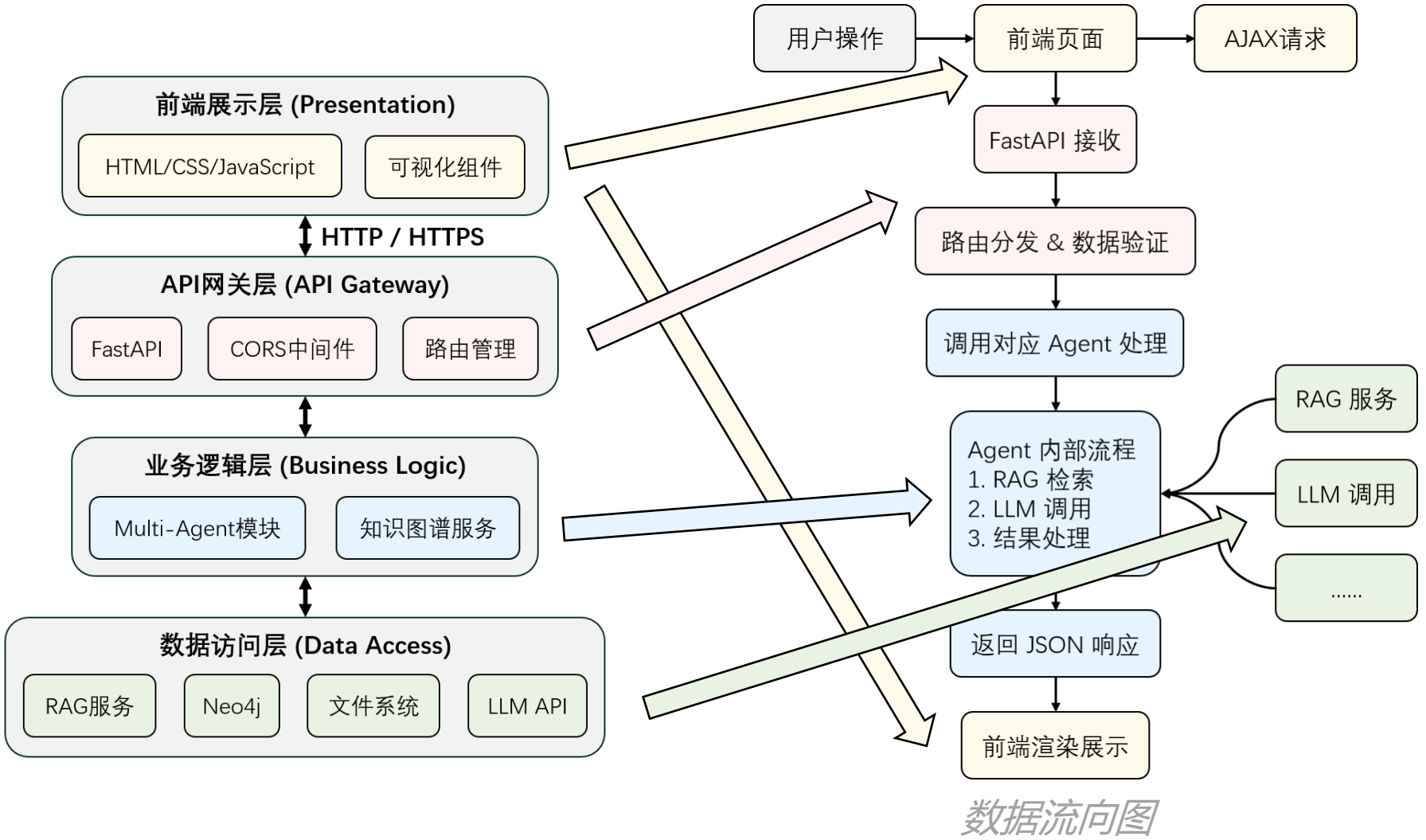
多Agent架构设计图

2.4 系统架构设计：前后端集成架构



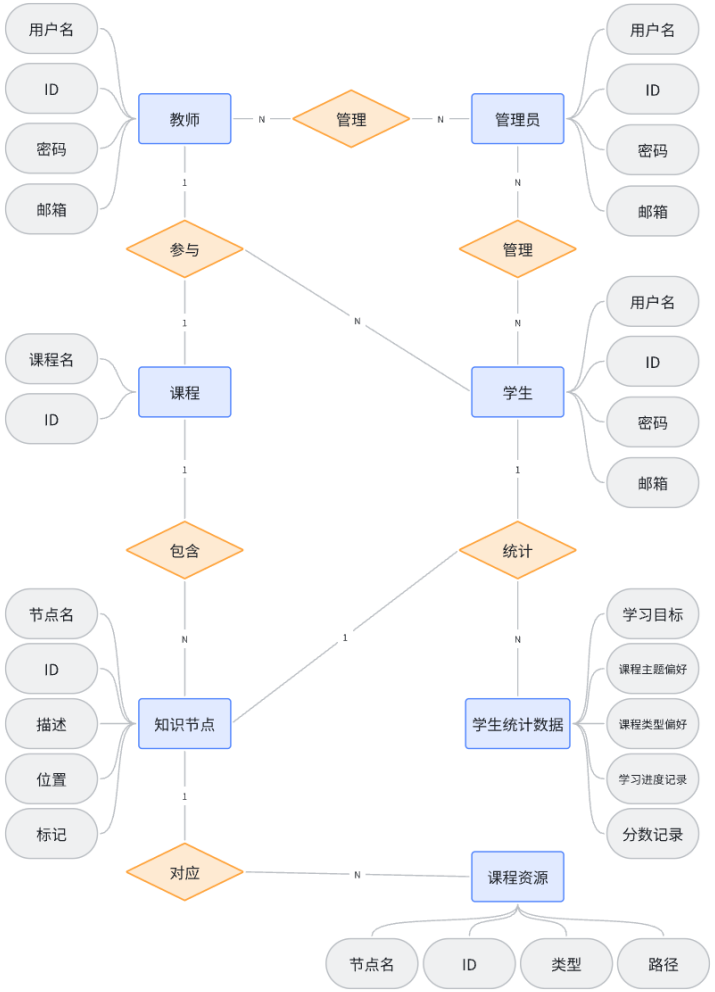
整体架构层次

2.4 系统架构设计：前后端集成架构

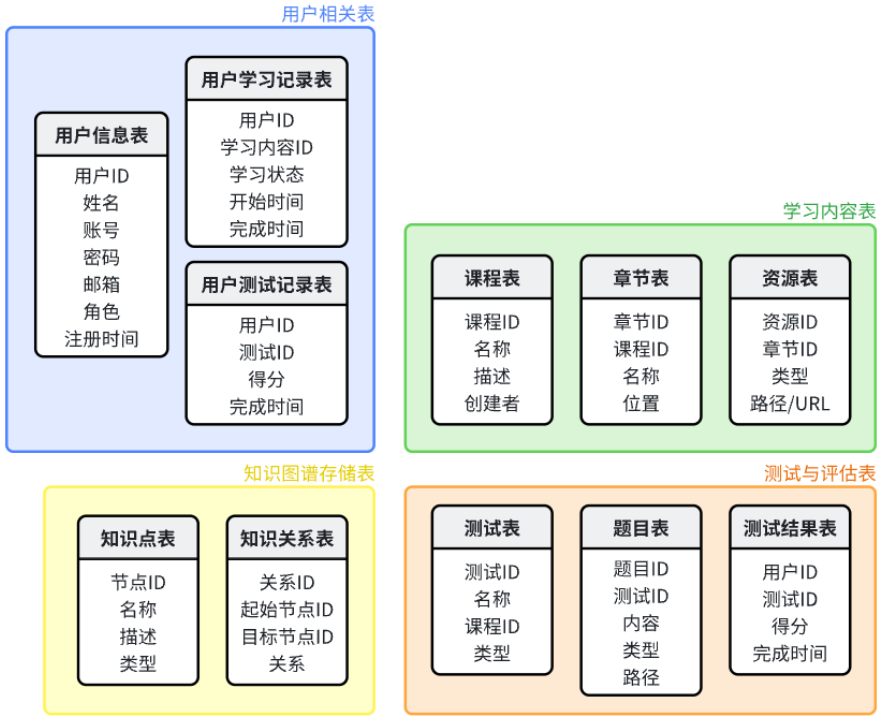


数据流向图

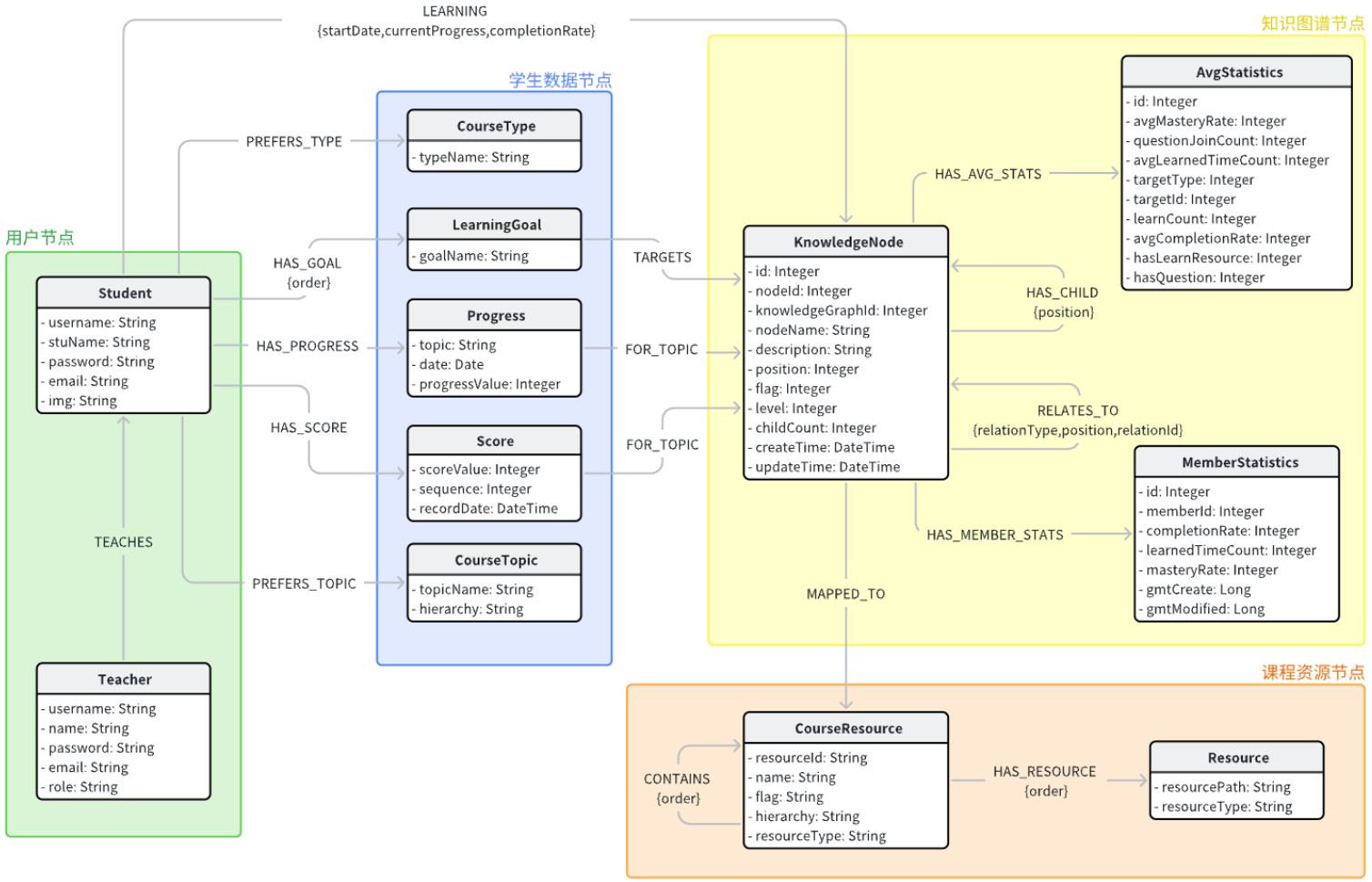
系统架构设计：数据层设计



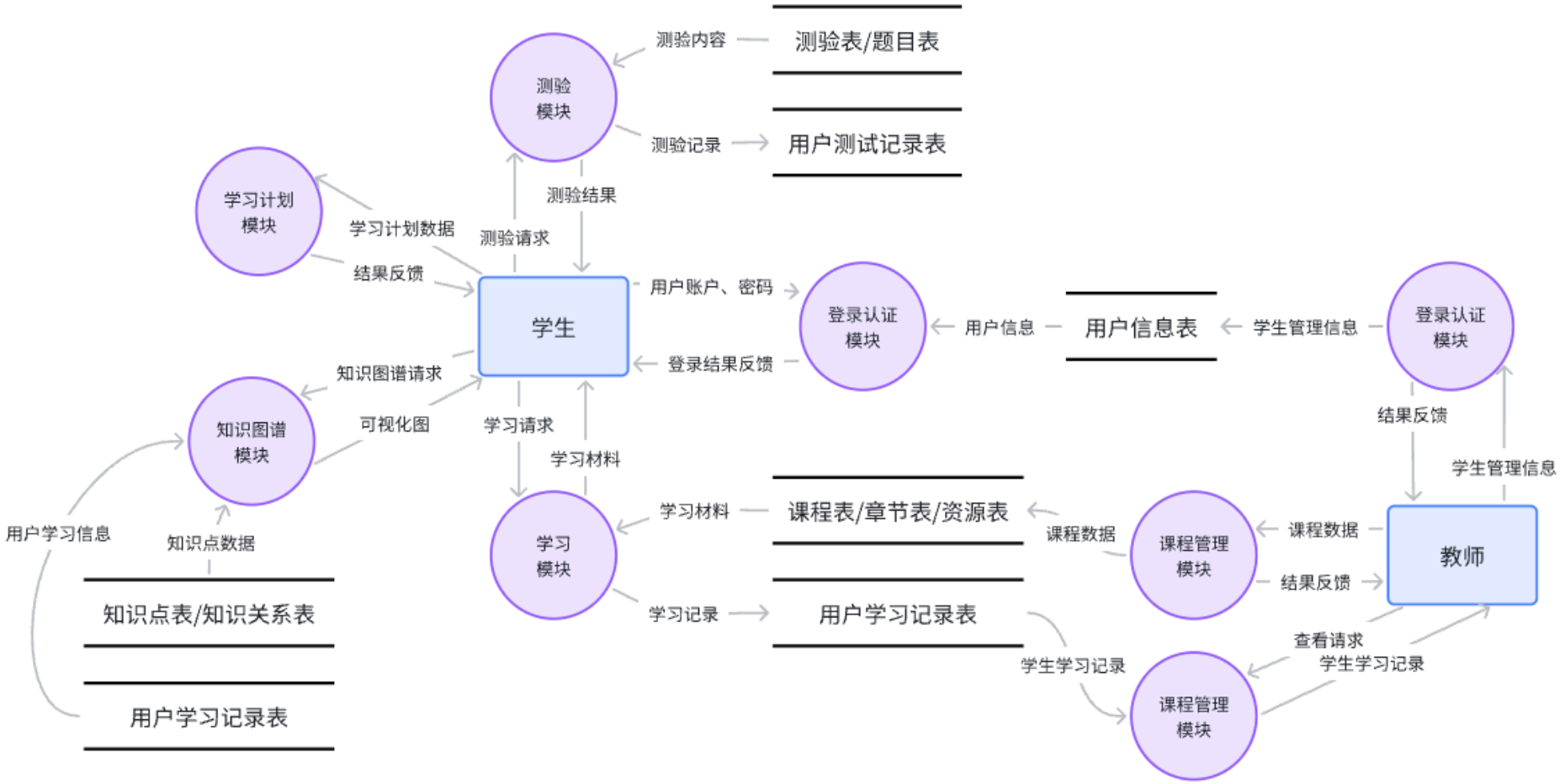
整体 ER 图



主要数据表结构


LEARNING
{startDate,currentProgress,completionRate}
用户节点
学生数据节点
知识图谱节点
课程资源节点
Student
- username: String
- stuName: String
- password: String
- email: String
- img: String
Teacher
- username: String
- name: String
- password: String
- email: String
- role: String
TEACHES
PREFERS_TYPE
HAS_GOAL
{order}
HAS_PROGRESS
HAS_SCORE
PREFERS_TOPIC
CourseType
- typeName: String
LearningGoal
- goalName: String
Progress
- topic: String
- date: Date
- progressValue: Integer
Score
- scoreValue: Integer
- sequence: Integer
- recordDate: DateTime
CourseTopic
- topicName: String
- hierarchy: String
TARGETS
FOR_TOPIC
FOR_TOPIC
KnowledgeNode
- id: Integer
- nodeId: Integer
- knowledgeGraphId: Integer
- nodeName: String
- description: String
- position: Integer
- flag: Integer
- level: Integer
- childCount: Integer
- createTime: DateTime
- updateTime: DateTime
HAS_AVG_STATS
AvgStatistics
- id: Integer
- avgMasteryRate: Integer
- questionJoinCount: Integer
- avgLearnedTimeCount: Integer
- targetType: Integer
- targetId: Integer
- learnCount: Integer
- avgCompletionRate: Integer
- hasLearnResource: Integer
- hasQuestion: Integer
HAS_CHILD
{position}
RELATES_TO
{relationType,position,relationId}
HAS_MEMBER_STATS
MemberStatistics
- id: Integer
- memberId: Integer
- completionRate: Integer
- learnedTimeCount: Integer
- masteryRate: Integer
- gmtCreate: Long
- gmtModified: Long
MAPPED_TO
CONTAINS
{order}
CourseResource
- resourceId: String
- name: String
- flag: String
- hierarchy: String
- resourceType: String
HAS_RESOURCE
{order}
Resource
- resourcePath: String
- resourceType: String

数据库类图


测验内容
测验表/题目表
测验模块
测验记录
用户测试记录表
测验结果
学习计划模块
学习计划数据
测验请求
结果反馈
学生
用户账户、密码
登录认证模块
用户信息
用户信息表
学生管理信息
登录认证模块
登录结果反馈
知识图谱请求
知识图谱模块
可视化图
学习请求
学习材料
结果反馈
学生管理信息
用户学习信息
知识点数据
学习材料
课程表/章节表/资源表
课程数据
课程管理模块
课程数据
教师
学习模块
学习记录
用户学习记录表
结果反馈
查看请求
学生学习记录
学生学习记录
知识点表/知识关系表
课程管理模块
用户学习记录表

数据流转示意图

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

章节内容

I. 知识图谱
- 作用范围标注图
- 应用流程图
- 数据结构示意图
- 图数据库关系示意图

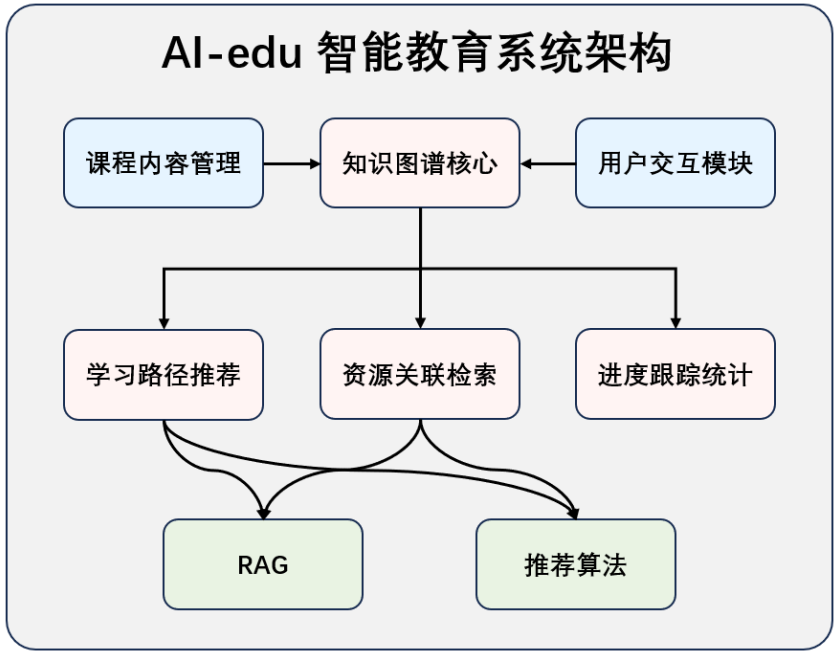
II. RAG 技术
- 作用范围标注图
- 应用流程图
- 详细技术流程图
- 关键参数配置图标
- 应用场景

III. 模型微调
- 数据准备流程图、模型微调流程图
- 应用架构
- 参数选择
- 提升对比

IV. 个性化推荐算法
- 总体流程图
- 完整流程

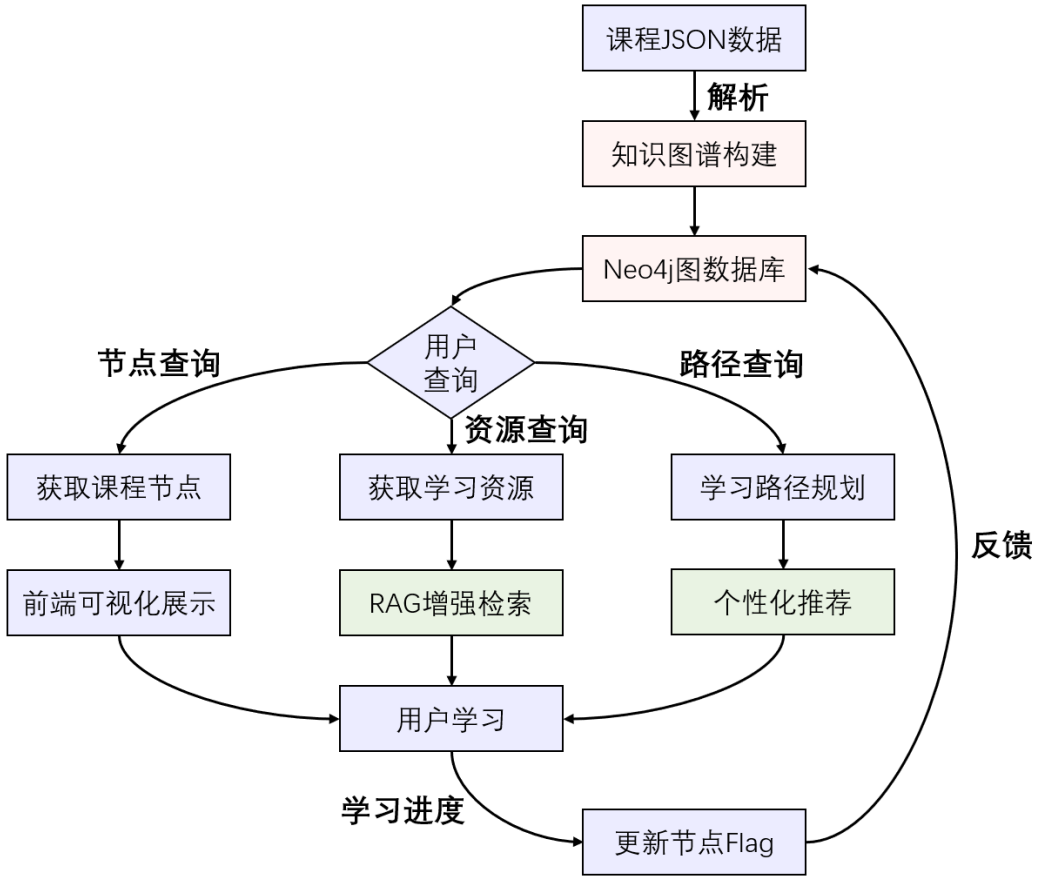
这里介绍**核心技术**实现，
锦上添花的**易实现功能**就不再赘述。
（如 EasyOCR 等）

核心技术实现：知识图谱

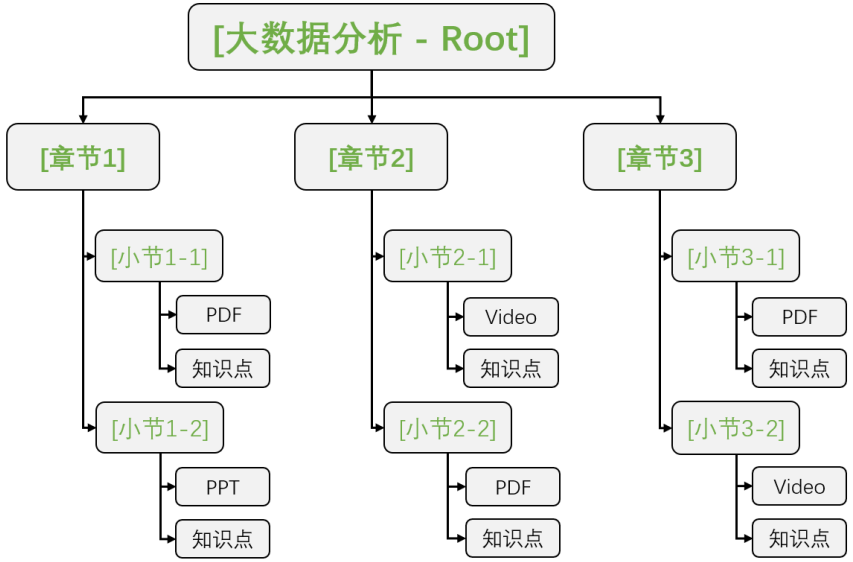


知识图谱作用范围标注图

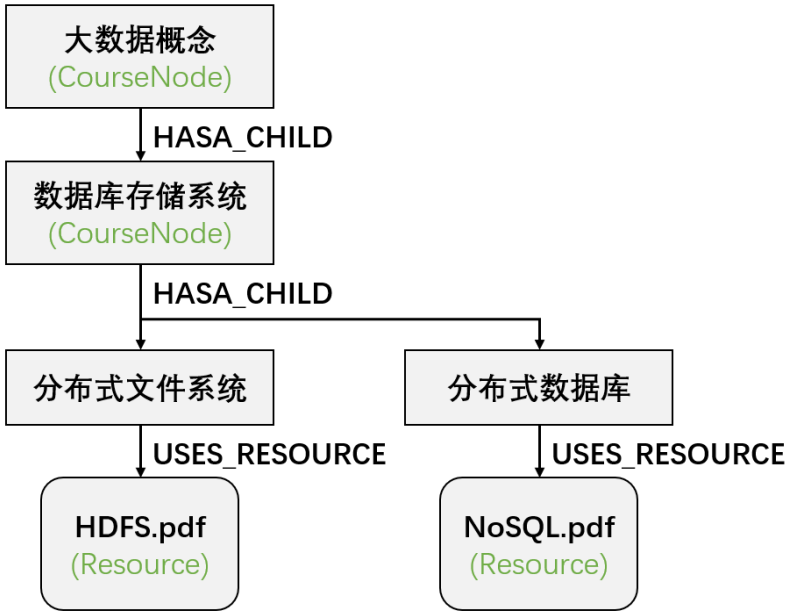
粉色: 知识图谱直接作用区域
蓝色: 知识图谱间接影响区域
绿色: 与知识图谱协同工作模块



知识图谱应用流程图

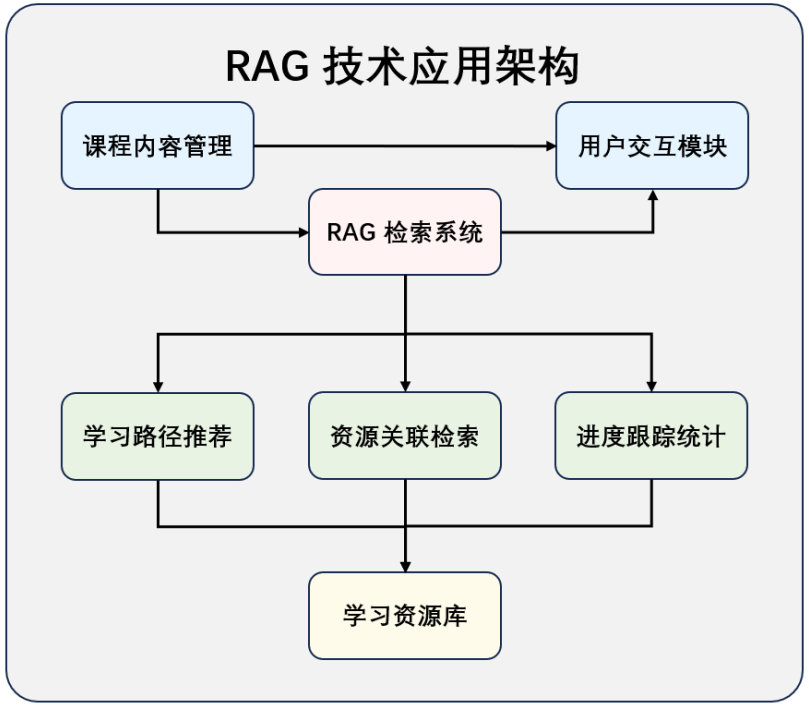


知识图谱数据结构示意图



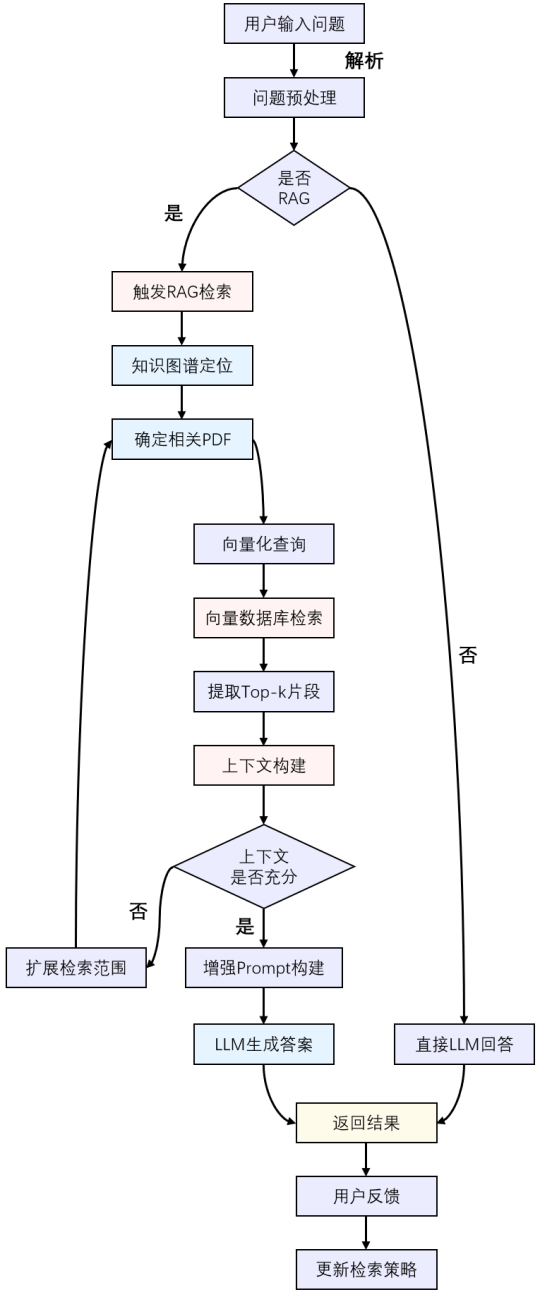
Neo4j 图数据库关系示意图

核心技术实现：RAG 技术

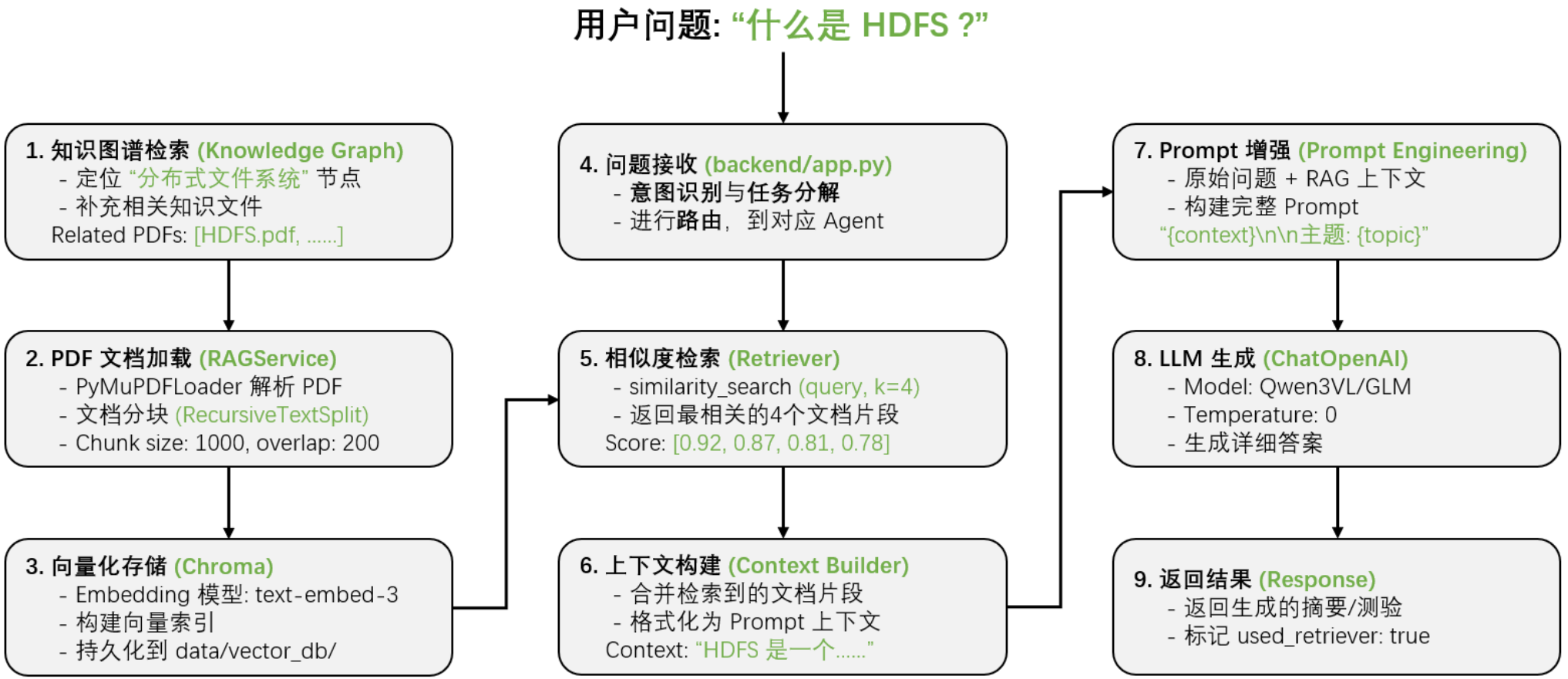


RAG 作用范围标注图

粉色: RAG核心检索模块 (直接作用区)
蓝色: 用户交互层 (数据输入输出)
绿色: 底层支撑技术 (间接支持)
黄色: 数据资源层 (被检索对象)



RAG 应用流程图

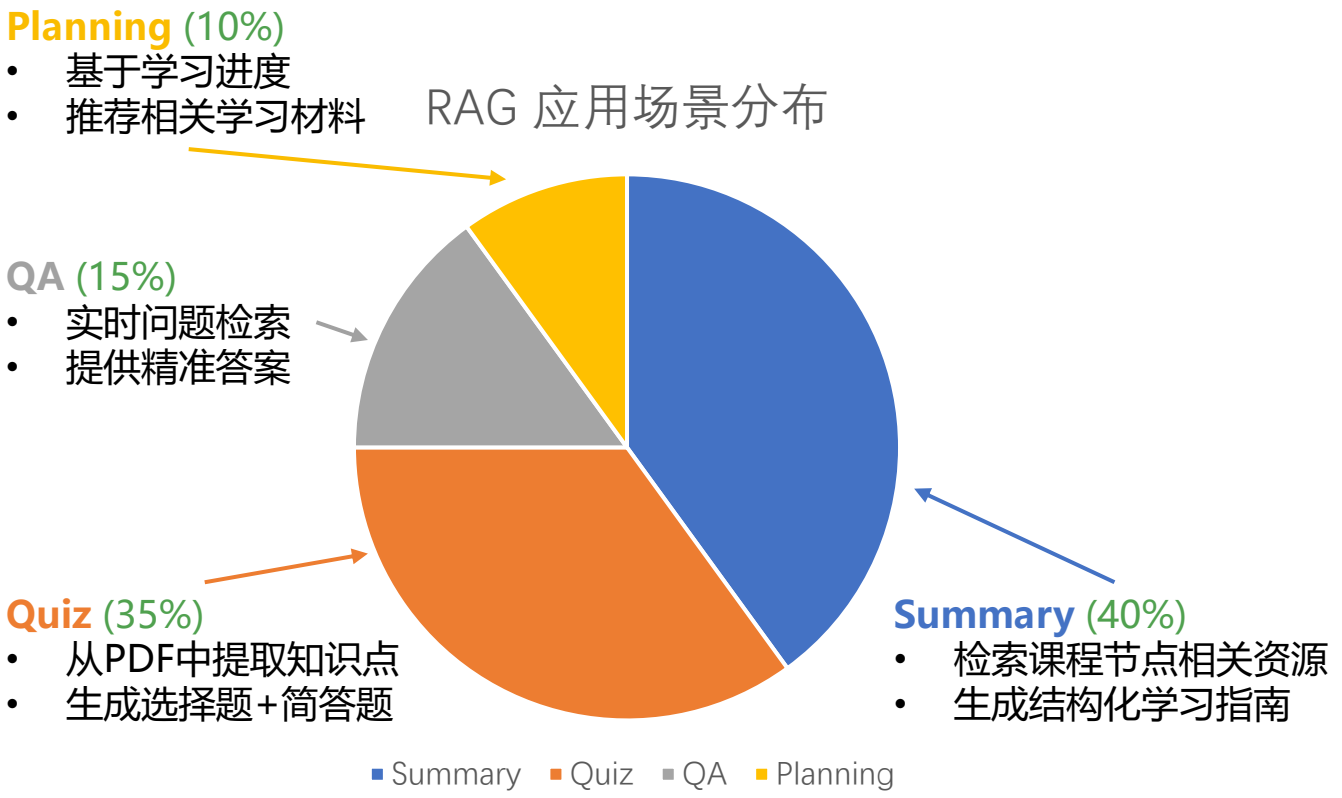

用户问题: “什么是 HDFS ?”
1. 知识图谱检索 (Knowledge Graph)
- 定位 “分布式文件系统” 节点
- 补充相关知识文件
Related PDFs: [HDFS.pdf, ……]
2. PDF 文档加载 (RAGService)
- PyMuPDFLoader 解析 PDF
- 文档分块 (RecursiveTextSplit)
- Chunk size: 1000, overlap: 200
3. 向量化存储 (Chroma)
- Embedding 模型: text-embed-3
- 构建向量索引
- 持久化到 data/vector_db/
4. 问题接收 (backend/app.py)
- 意图识别与任务分解
- 进行路由，到对应 Agent
5. 相似度检索 (Retriever)
- similarity_search (query, k=4)
- 返回最相关的4个文档片段
Score: [0.92, 0.87, 0.81, 0.78]
6. 上下文构建 (Context Builder)
- 合并检索到的文档片段
- 格式化为 Prompt 上下文
Context: “HDFS 是一个……”
7. Prompt 增强 (Prompt Engineering)
- 原始问题 + RAG 上下文
- 构建完整 Prompt
“{context}\n\n主题: {topic}”
8. LLM 生成 (ChatOpenAI)
- Model: Qwen3VL/GLM
- Temperature: 0
- 生成详细答案
9. 返回结果 (Response)
- 返回生成的摘要/测验
- 标记 used_retriever: true

RAG详细技术流程图

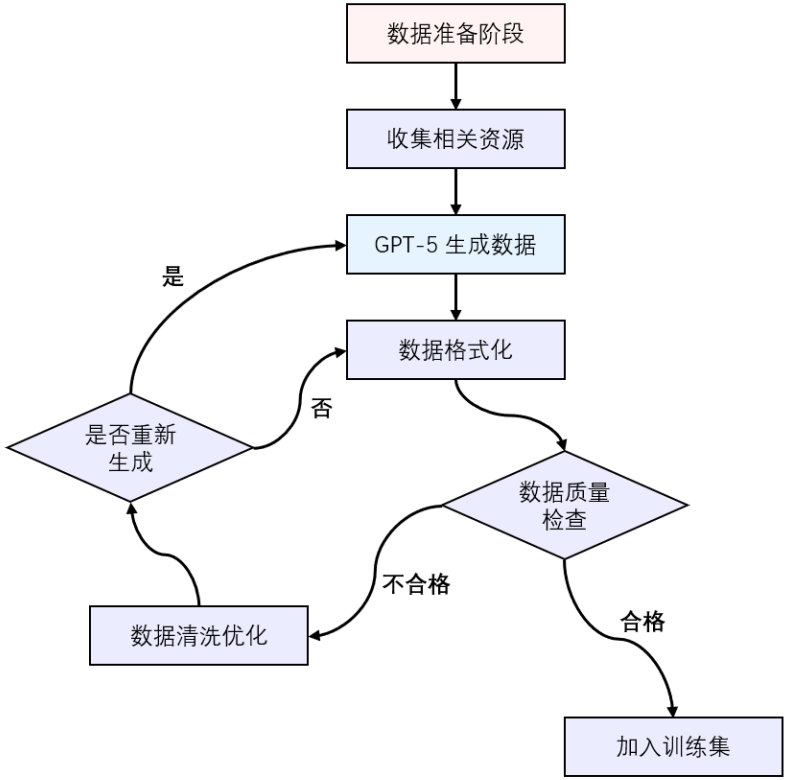
核心技术实现：RAG 技术

配置项	参数值	说明
文档分块大小	1000	Chunk_size
文档重叠	200	Chunk_overlap
检索文档数	4	Top_k
Embedding 维度	1536	text-embedding-3
向量数据库	Chroma	持久化存储
相似度算法	Cosine	余弦相似度

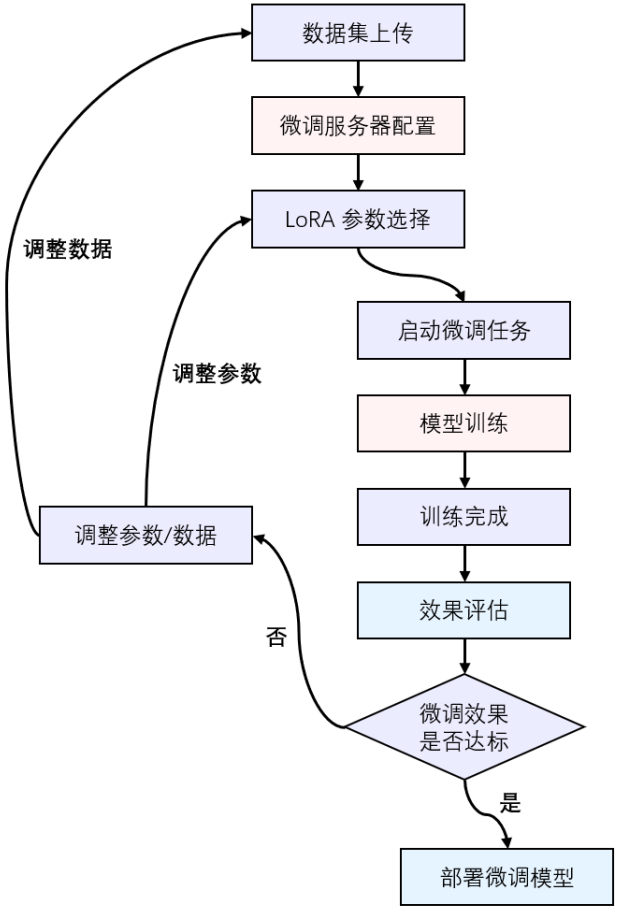
RAG关键参数配置图表



RAG 在系统中的应用场景



数据准备流程图



模型微调流程图

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

构造 500 个 QA 的训练数据

```
tools/
└── fine_tune/
    ├── __init__.py
    ├── qa_generator.py          # 核心生成器
    ├── qa_processor.py          # 数据处理与验证
    └── prompts.py               # Prompt模板库
data/
└── fine_tune/
    ├── raw_qa/                  # 原始生成的QA对
    ├── processed_qa/            # 处理后的训练数据
    └── knowledge_base/          # 课程知识片段
```

代码结构

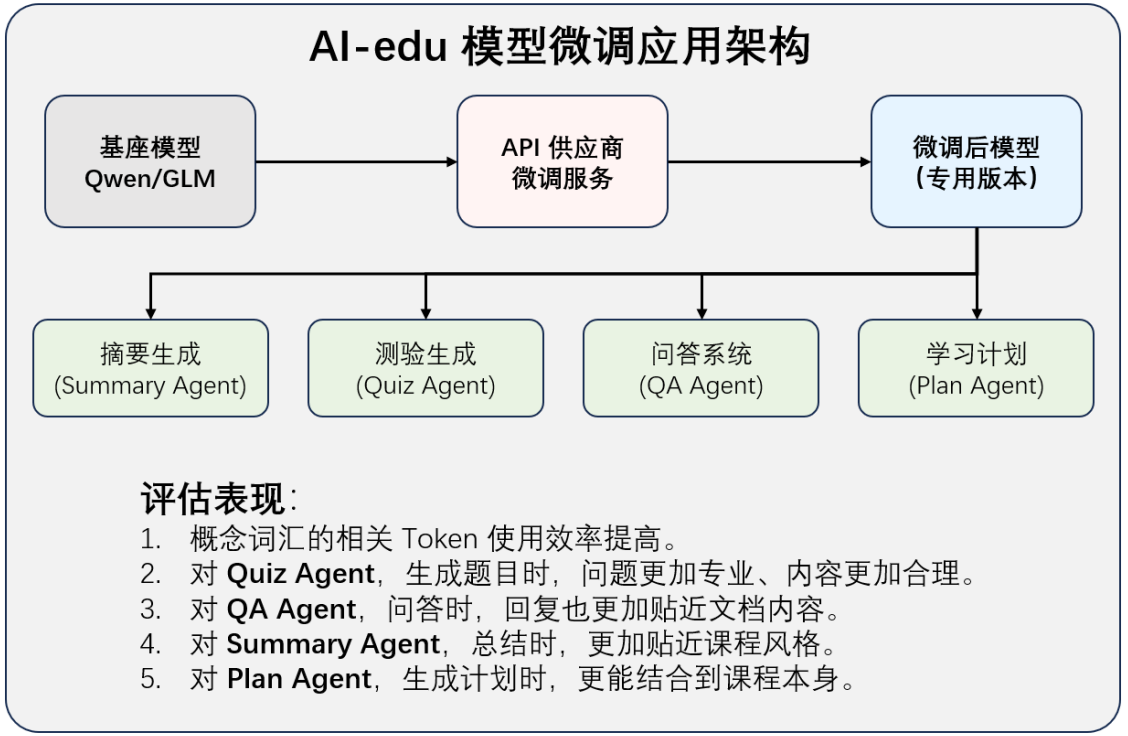
```
{"messages": [{"role": "system", "content": "你是一位大数据分析课程的专业助教..."}, {"role": "user", "content": "什么是MapReduce？"}, {"role": "assistant", "content": "MapReduce是一种分布式计算模型..."}]}
{"messages": [{"role": "system", "content": "你是一位大数据分析课程的专业助教..."}, {"role": "user", "content": "HDFS的容错机制是如何实现的？"}, {"role": "assistant", "content": "HDFS通过数据副本机制实现容错..."}]}
```

training_data.json

```
{
 "messages": [
  {
   "role": "system",
   "content": "你是一位大数据分析课程的专业助教，请基于课程知识准确回答学生问题。"
  },
  {
   "role": "user",
   "content": "什么是MapReduce？它的核心思想是什么？"
  },
  {
   "role": "assistant",
   "content": "MapReduce是由Google提出的一种分布式计算模型，用于处理大规模数据集。它的核心思想是将复杂的并行计算任务分解为Map（映射）和Reduce（归约）两个阶段：\n\n1. **Map阶段**：将输入数据分割成独立的块，并行处理后生成中间键值对（key-value pairs）。每个Map任务处理一部分数据，互不干扰。\n\n2. **Reduce阶段**：对Map阶段产生的中间结果按照键（key）进行分组，然后对每组数据执行归约操作，最终生成输出结果。\n\n这种分而治之的策略使得MapReduce能够在数千台服务器上并行处理PB级别的数据，具有高扩展性和容错性。典型应用包括日志分析、索引构建、数据统计等场景。"
  }
 ]
}
```

具体 QA 示例

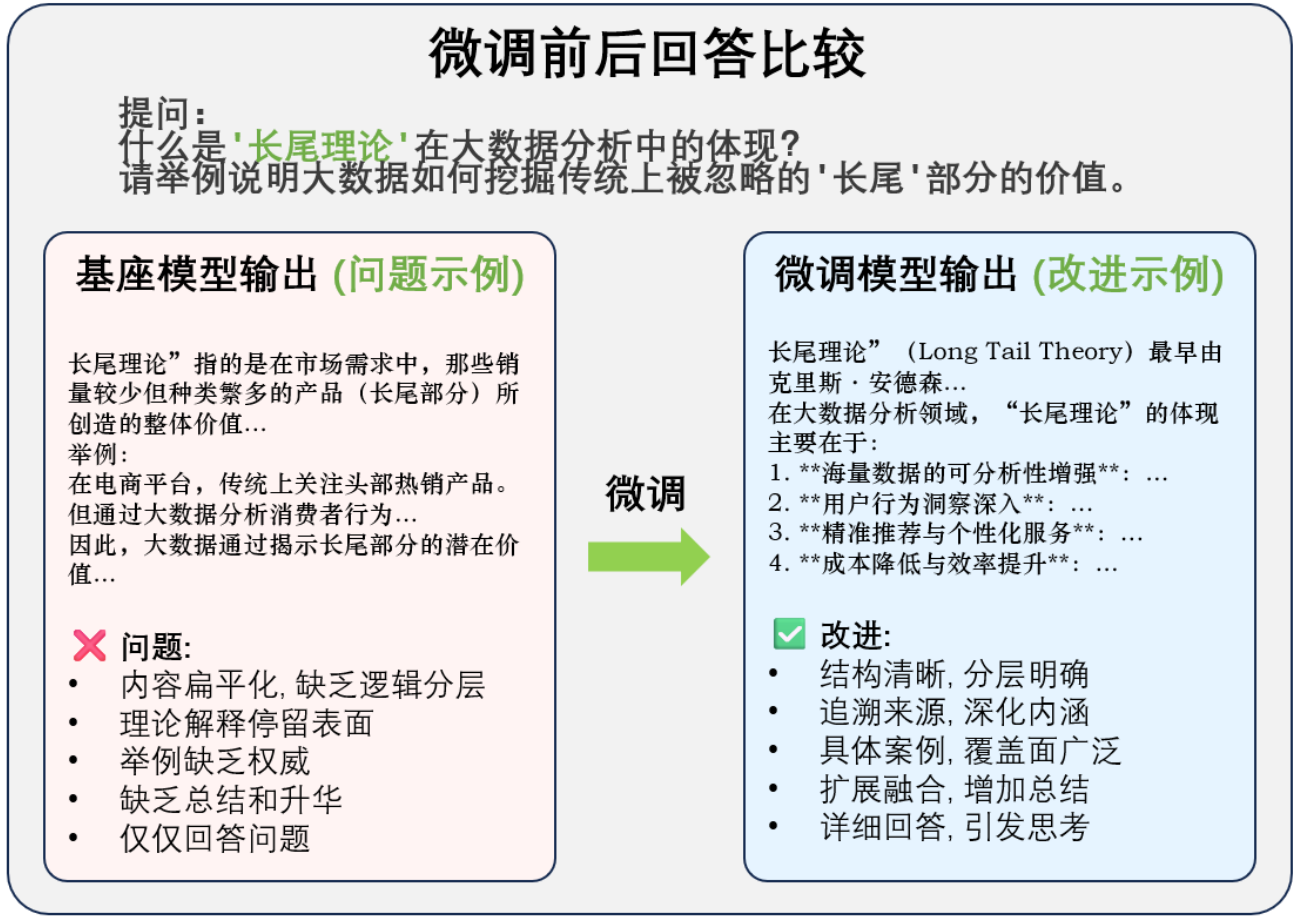
核心技术实现：模型微调



微调模型应用架构

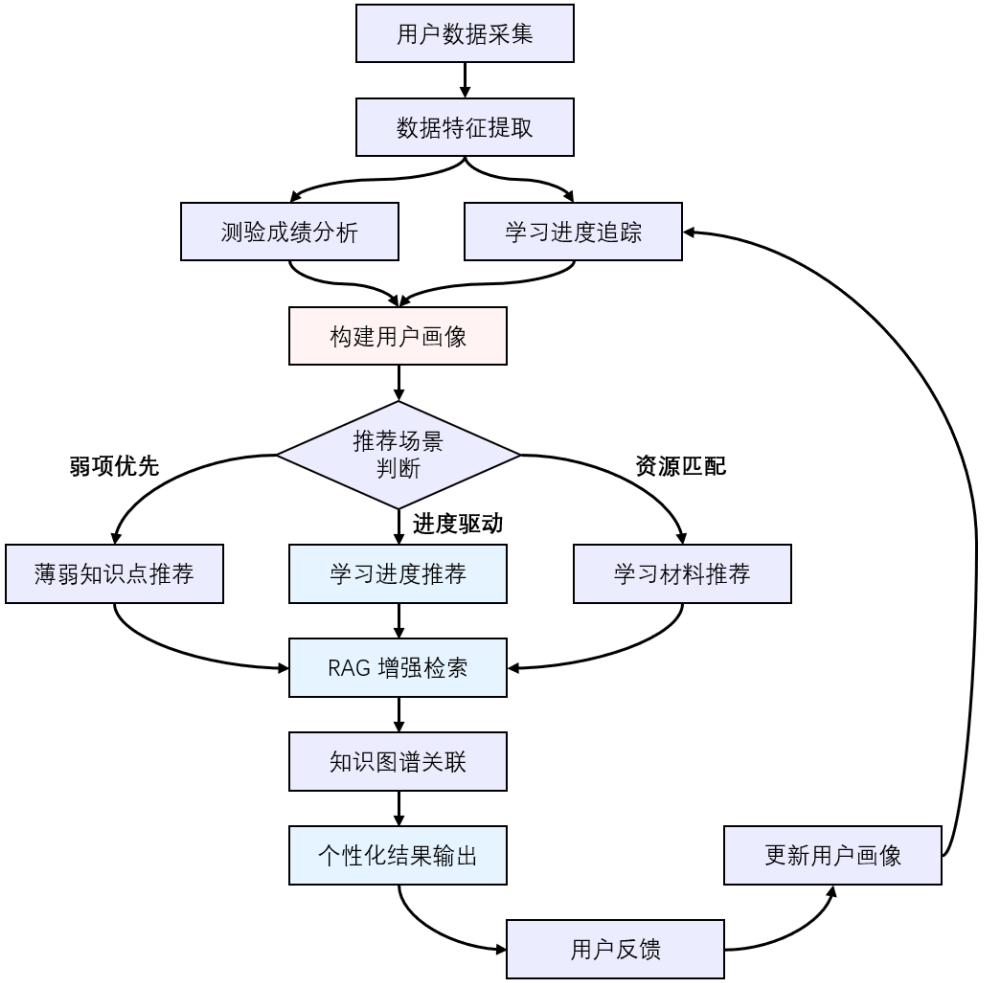
参数名	数值
Learning Rate	0.0001
Number of Epochs	3
Batch Size	8
LoRA Rank	8
LoRA Alpha	32
LoRA Dropout	0.05

微调参数选择


微调前后回答比较
提问：
什么是'长尾理论'在大数据分析中的体现？
请举例说明大数据如何挖掘传统上被忽略的'长尾'部分的价值。
基座模型输出 (问题示例)
长尾理论”指的是在市场需求中，那些销量较少但种类繁多的产品（长尾部分）所创造的整体价值…
举例：
在电商平台，传统上关注头部热销产品。但通过大数据分析消费者行为…
因此，大数据通过揭示长尾部分的潜在价值…
问题:
内容扁平化，缺乏逻辑分层
理论解释停留表面
举例缺乏权威
缺乏总结和升华
仅仅回答问题
微调
微调模型输出 (改进示例)
长尾理论”（Long Tail Theory）最早由克里斯·安德森…
在大数据分析领域，“长尾理论”的体现主要在于：
1. **海量数据的可分析性增强**：…
2. **用户行为洞察深入**：…
3. **精准推荐与个性化服务**：…
4. **成本降低与效率提升**：…
改进:
结构清晰，分层明确
追溯来源，深化内涵
具体案例，覆盖面广泛
扩展融合，增加总结
详细回答，引发思考

提升对比

3.4 核心技术实现：个性化推荐算法



算法总体流程图

2025-11-10 基础知识

掌握大数据分析，通过考试

2025-11-10

主题: 掌握大数据分析，通过考试

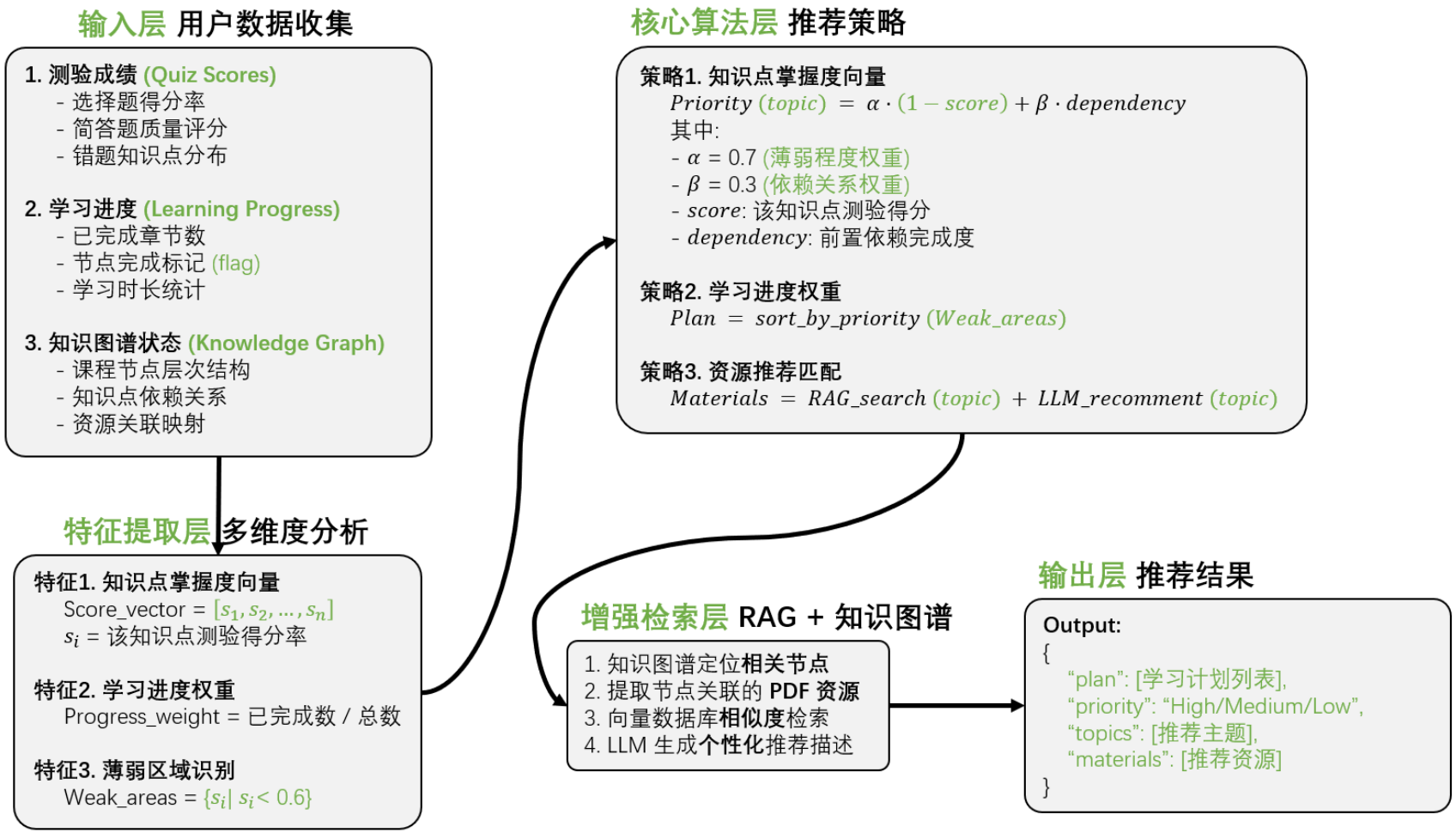
学习类型: 基础知识

截止日期: 2025-11-17

推荐材料:

- 以下是为帮助您"掌握大数据分析，通过考试"而精心整理的推荐资源列表，涵盖核心理论、技术架构、实战应用与备考要点：
- **一、核心理论与技术基础**
- 1. **《大数据技术原理与应用》**
- 作者：林子雨
- 特点：系统讲解Hadoop、Spark、NoSQL等核心技术，配套实验指南，适合入门与考试重点梳理。
- 2. **《数据密集型应用系统设计》**
- 作者：Martin Kleppmann
- 重点章节：分布式数据系统、复制与分区、容错机制（如Raft/Paxos对比），深入理解分布式系统底层逻辑。
- 3. **论文《Google File System》及《MapReduce》**
- 来源：Google Research
- 意义：掌握GFS架构、分块策略、容错机制等经典设计思想，是考试常见案例题来源。
- **二、分布式存储与容错实战**
- 1. **《Hadoop权威指南》**
- 作者：Tom White

核心技术实现：个性化推荐算法



AI-edu 个性化推荐算法完整流程

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

章节内容

I. AI 内容检查评估机制

算法总体流程图
质量检测展开图

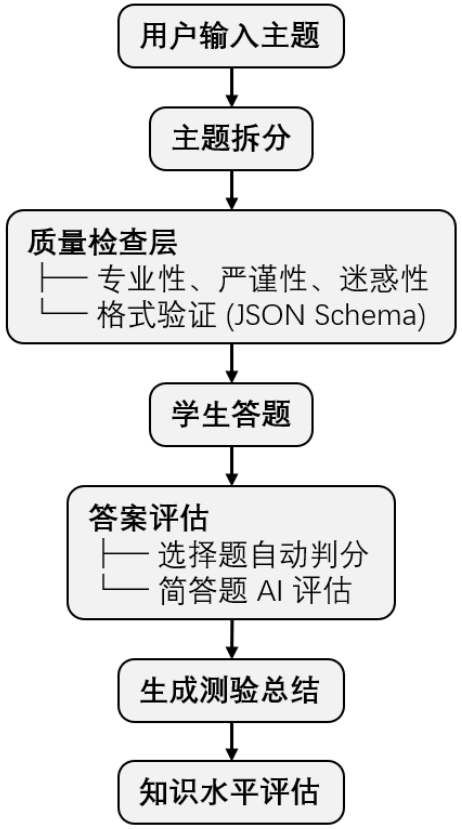
II. 测试题型覆盖

题型覆盖图表

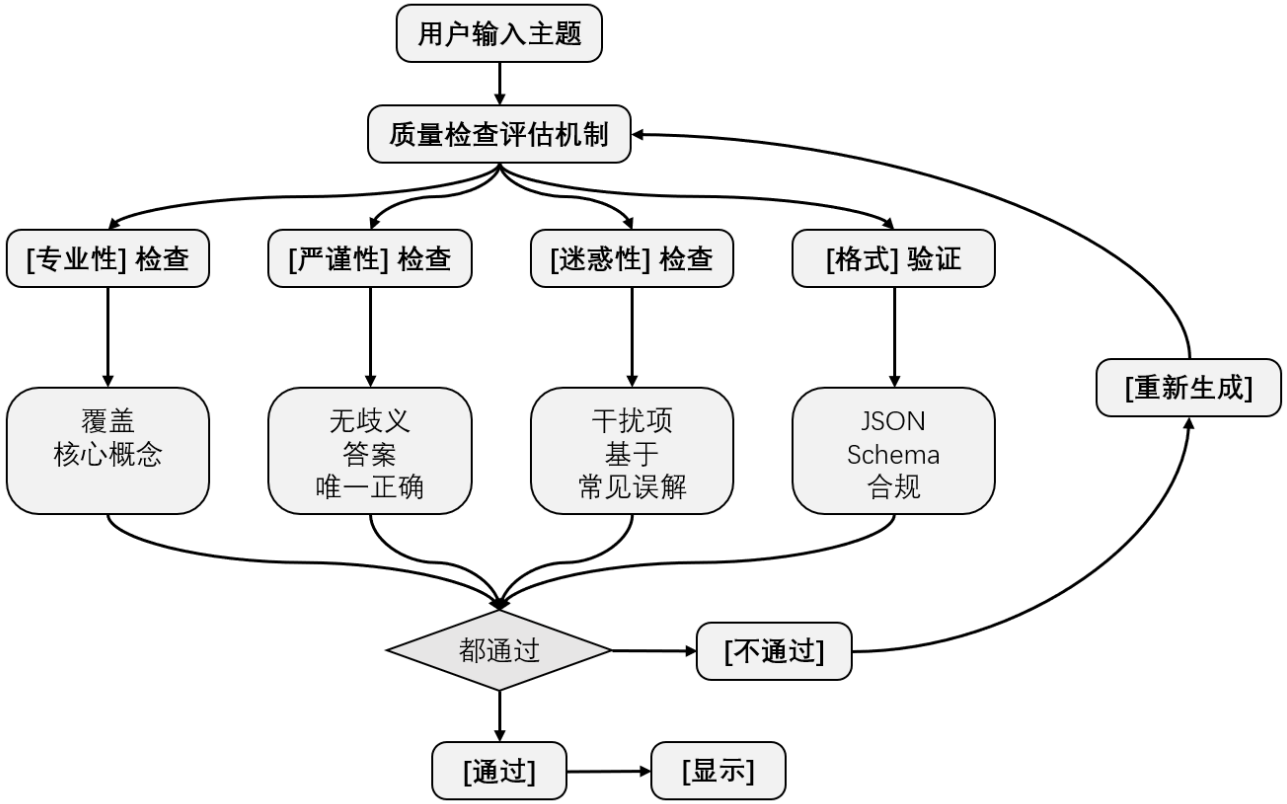
III. AI 内容日志审查机制

质量保证体系
指标效果图表
架构层级图

质量保障机制： AI 内容检查评估机制



算法总体流程图



质量检测展开图

4.2 质量保障机制： 测试题型覆盖

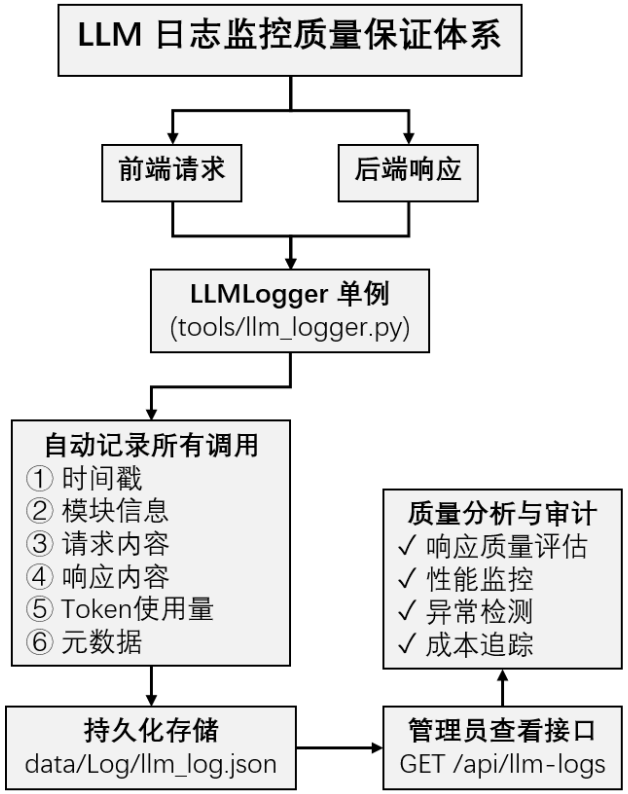
题型类别	数量配置	评估方式	应用场景
单选题 Single Choice	8题 / 测验 （80%）	自动判分 A/B/C/D 精确匹配	基础知识点考察 概念理解验证
简答题 Short Answer	2题 / 测验 （20%）	AI 评分 语义分析 综合评价	深度理解考察 应用能力评估

题型覆盖图表

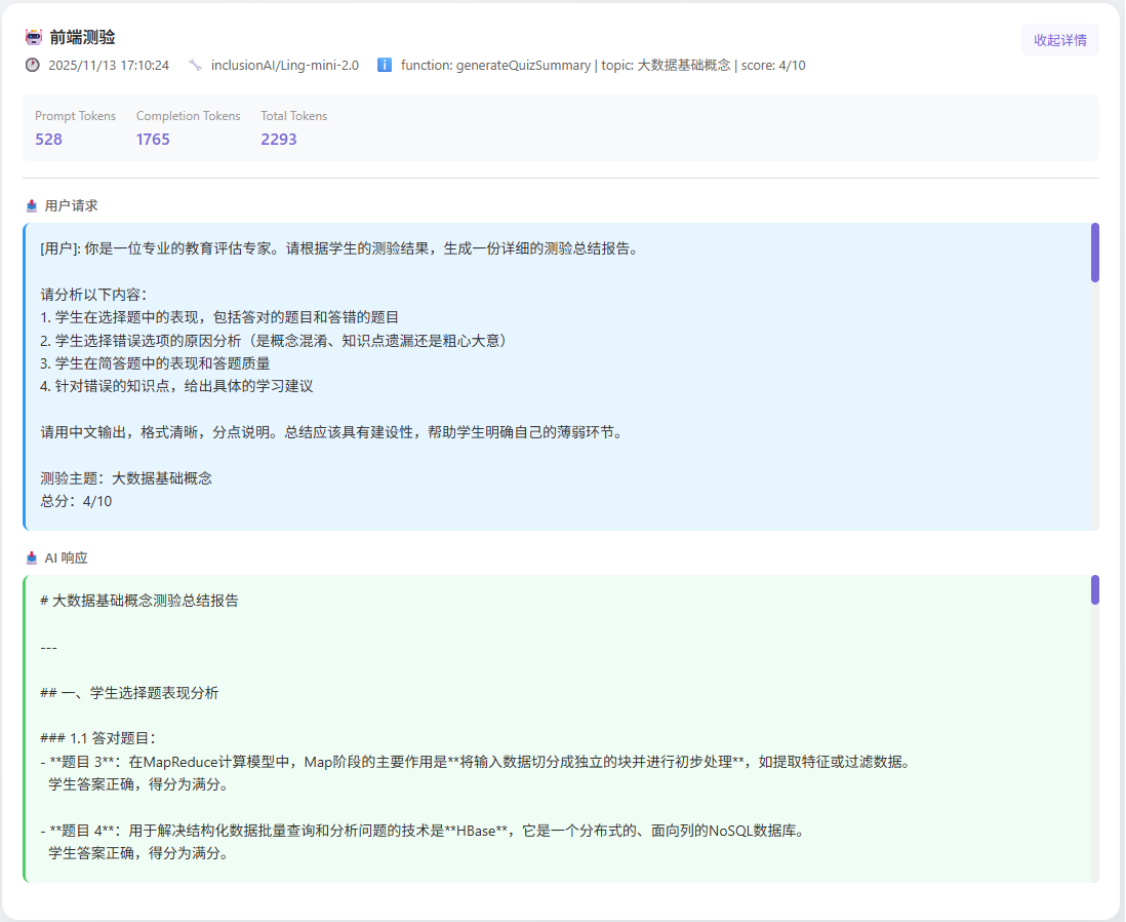




4.3 质量保障机制：AI 内容日志审查机制



LLM 日志监控质量保证体系

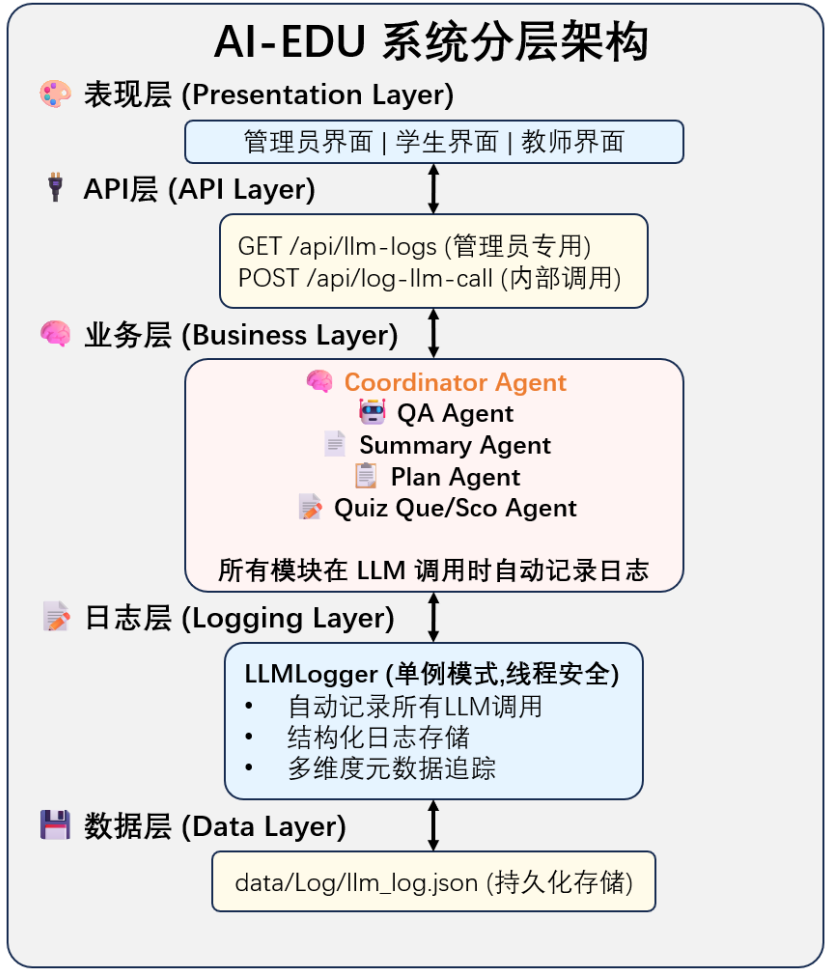


日志记录的详细字段展示

4.3 质量保障机制：AI 内容日志审查机制

功能维度	监控指标	保证效果
性能监控	Token 使用量 响应时间 调用频率	成本控制 性能优化 资源规划
质量审计	输入 Prompt 输出内容 完成原因	Prompt 优化 内容质量评估 异常识别
行为追踪	调用模块 业务场景 RAG 使用情况	用能使用分析 用户行为洞察 系统优化建议
异常检测	错误日志 Fallback 触发 异常响应	问题及时发现 系统稳定性保障 风险预警

LLM 日志监控指标效果图表



架构层级图

章节内容

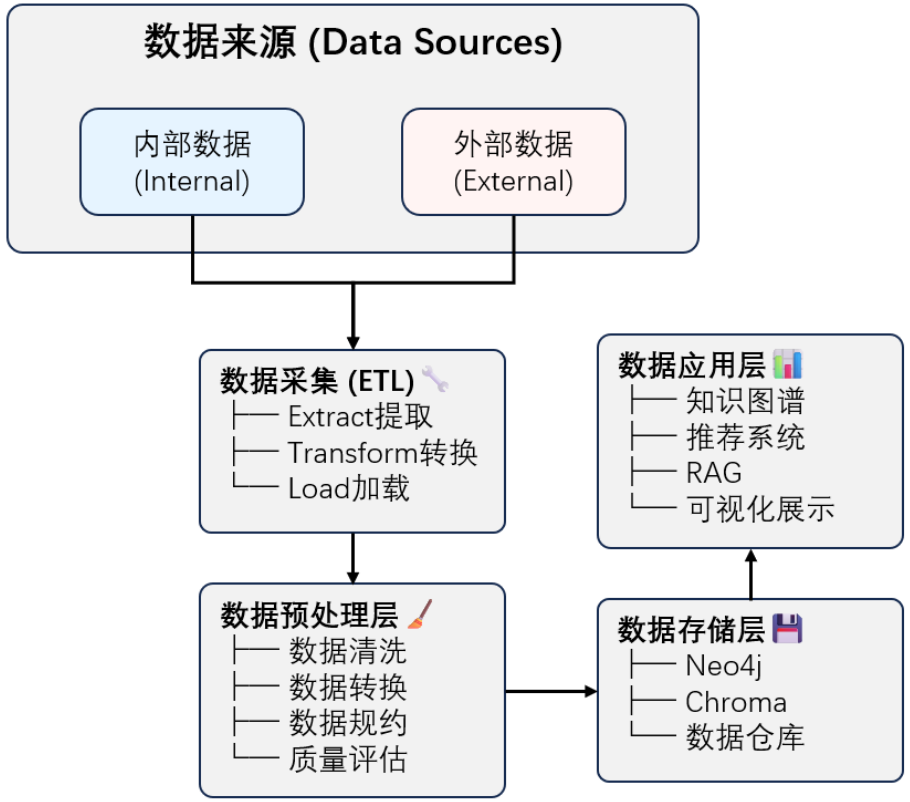
I. 数据制作与引用

总体数据来源图
各层数据来源图
数据流架构图

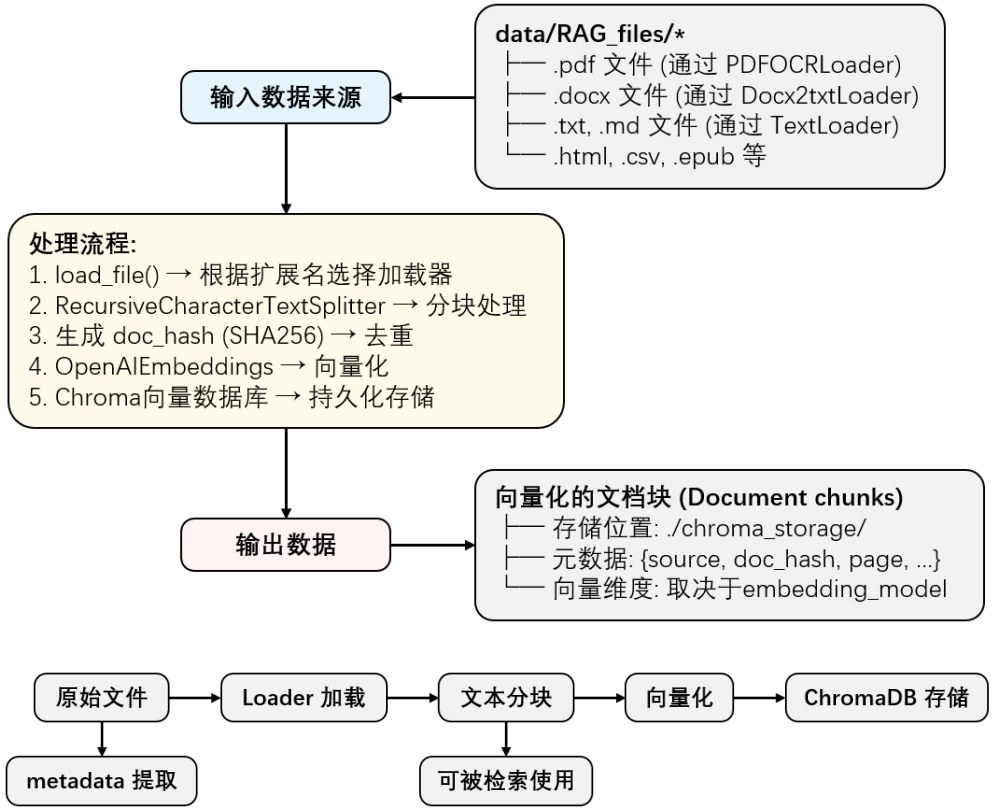
II. 前端页面逻辑

页面跳转图

数据与前端：数据制作与引用

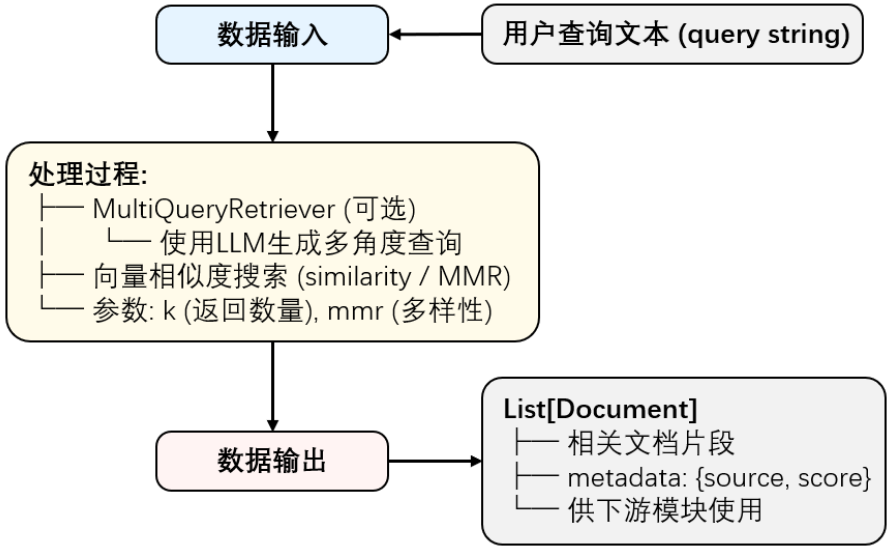


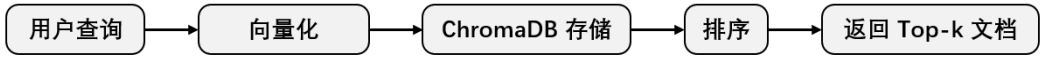
数据来源图



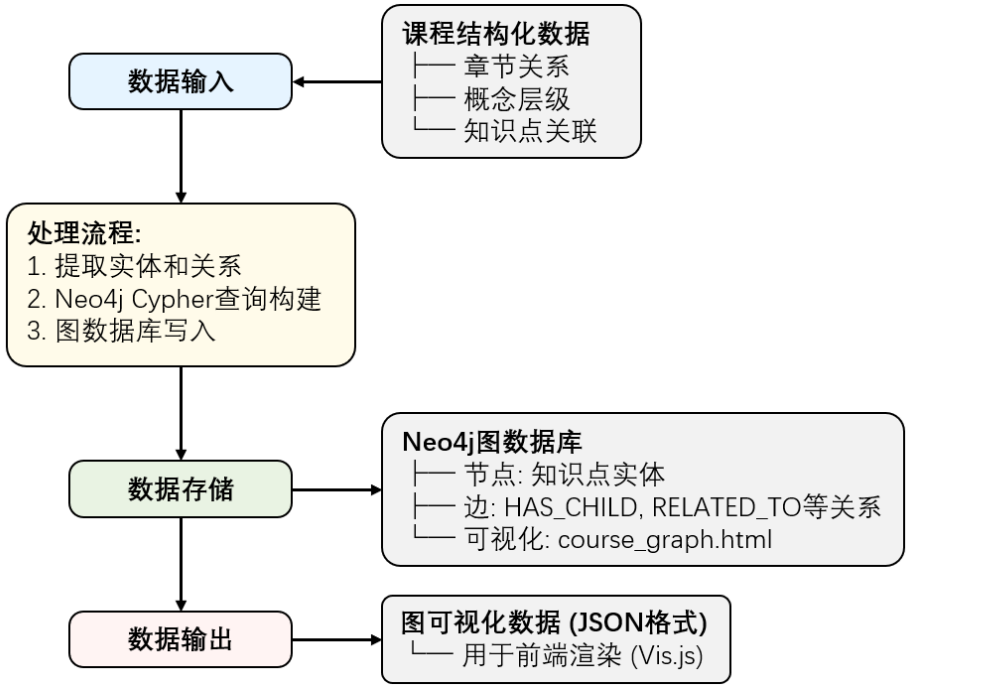
数据预处理层

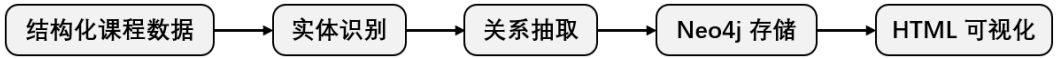
数据与前端：数据制作与引用





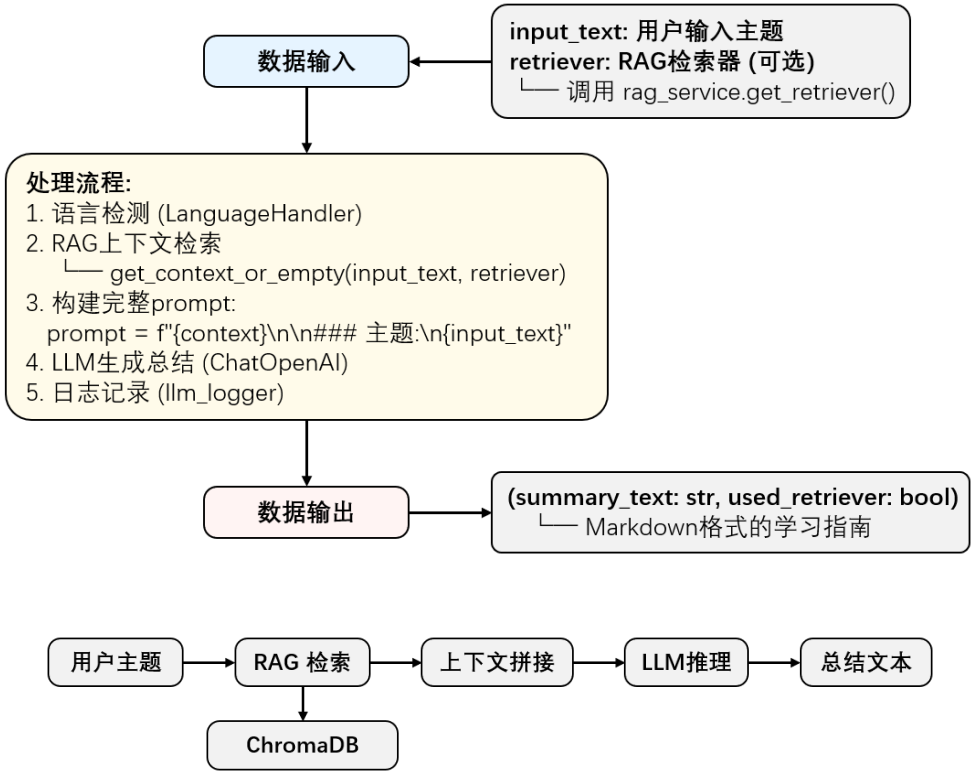
知识检索层



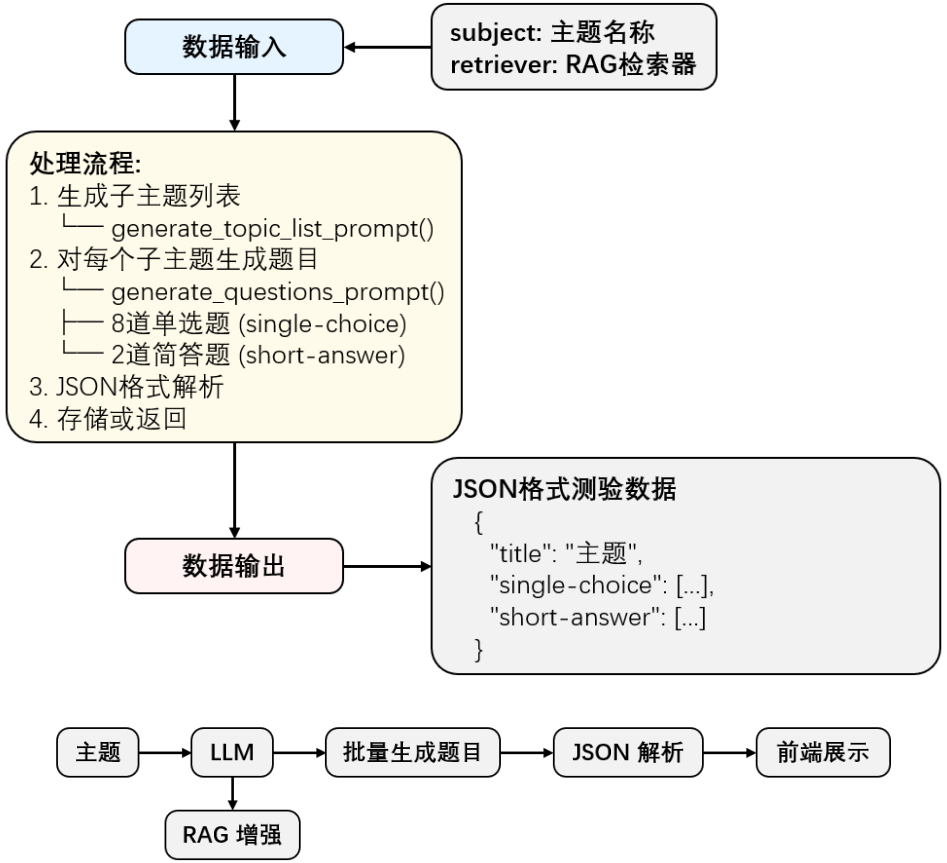


知识图谱层

数据与前端：数据制作与引用

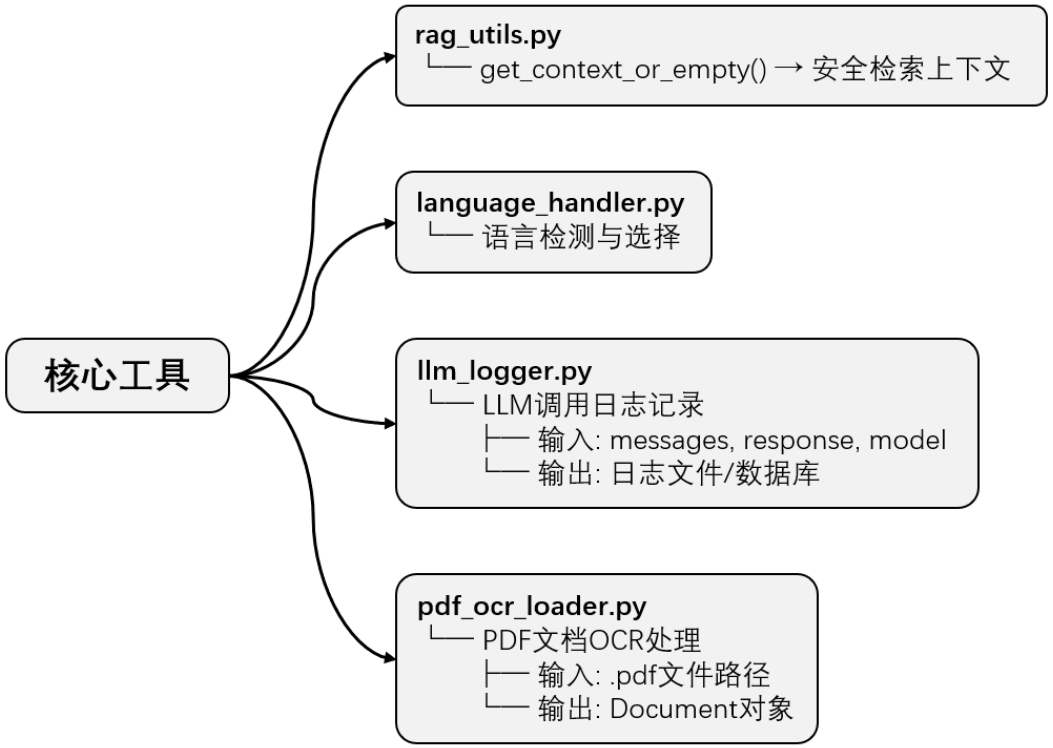


应用层 - 总结生成



应用层 – 测验生成

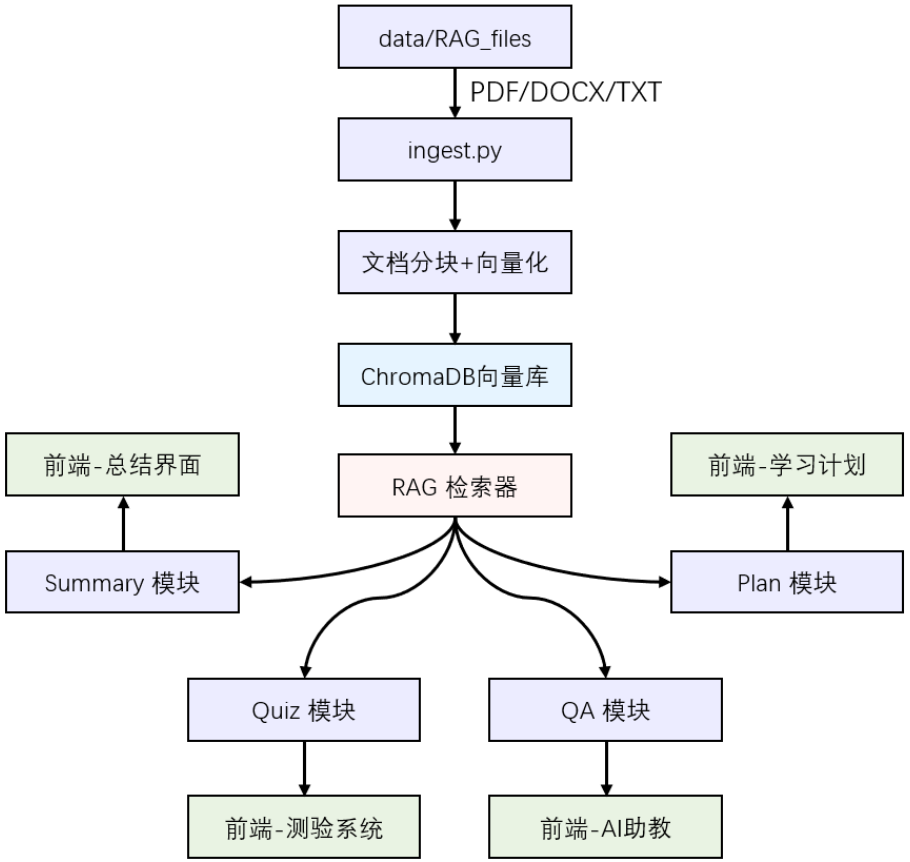
数据与前端：数据制作与引用



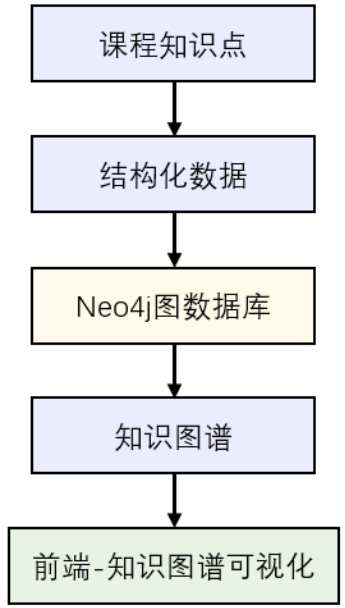
工具层

节点	输入格式	输出格式	存储位置
文档加载	PDF/DOCX/TXT	Document对象	内存
向量化	Document对象	向量+metadata	ChromaDB
图构建	课程结构数据	图节点+边	Neo4j
RAG检索	查询字符串	相关文档列表	内存
LLM推理	Prompt+上下文	生成文本	内存/日志
前端渲染	JSON/Markdown	HTML页面	浏览器

关键数据转换节点

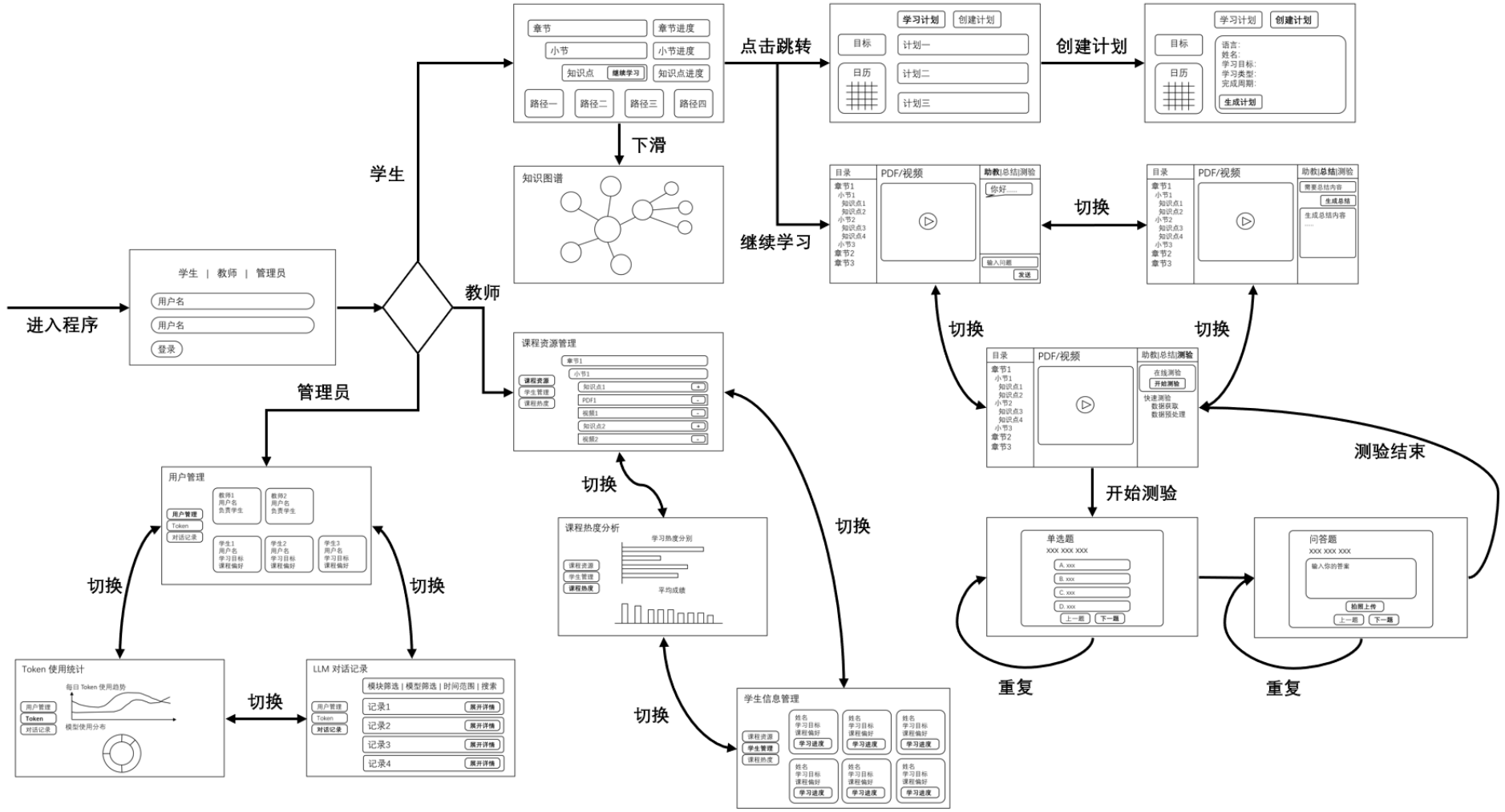


RAG 数据流架构图



知识图谱数据流架构图

数据与前端：前端页面逻辑



页面跳转图

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

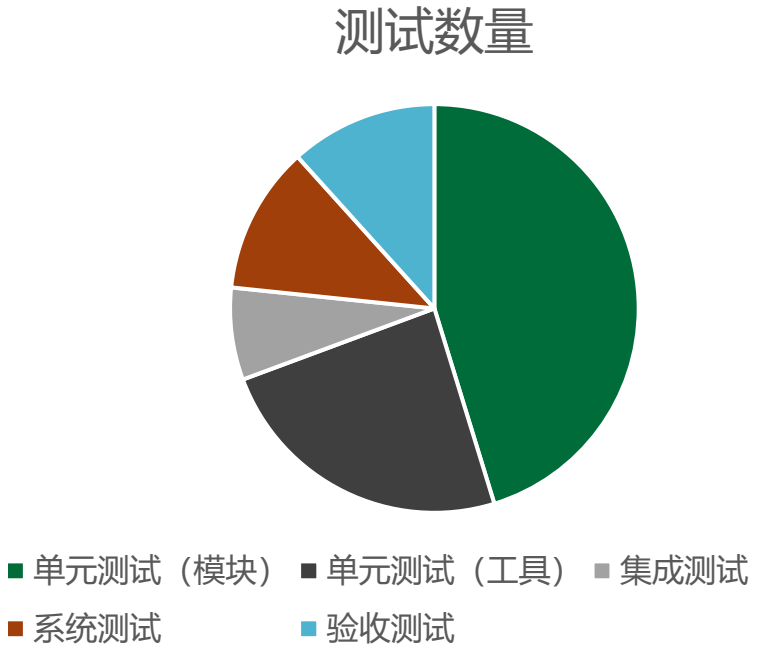
章节内容

I. **单元测试**
   模块
   工具
II. **集成测试**
III. **系统测试**
IV. **验收测试**

在每个 Sprint 的开发完成后，
我们都及时进行对应的**单元测试。**

对每个模块、工具进行拼装后，
我们也及时补充上**集成测试。**

最后我们进行了**系统测试**和**验收测试**，
形成了我们最终版本的 AI-EDU 系统。



总共 137 项

模块名	测试对象	测试方法	测试结果
edu_agent	rag_search	输入一个存在的查询，确保 RAGService 返回有效上下文	返回非空字符串，包含上下文，测试通过
		输入一个不存在的查询，确保处理无结果情况	返回空字符串或默认提示，测试通过
		模拟 RAGService 抛出异常	返回字符串以 "RAG search error:" 开头，测试通过
	create_agent	创建 AgentExecutor 对象	返回对象类型为 AgentExecutor，包含工具列表和 max_iterations=3，测试通过
		确认工具列表完整	工具列表包含 wikipedia_search、define_word、calculator、current_date、current_weekday、detect_language、rag_search，测试通过
	run_agent	输入 PDF 相关问题，模拟 RAGService 返回上下文	调用 rag_search 并返回上下文内容，测试通过
		输入问题包含已知上下文	直接返回最终答案，不再调用工具，测试通过
		输入问题语言为中文	LanguageHandler.choose_or_detect 返回 "zh"，最终答案为中文，测试通过
		模拟 Agent 执行抛出异常	捕获异常，返回 "Agent error:..." 字符串，测试通过
		return_details=True 参数	返回 (output, used_fallback, used_retriever) 元组，测试通过
	日志记录	模拟调用 run_agent	日志包含用户问题、检测语言、Agent/LLM 输出，测试通过
	异常情况	模拟 LLM 调用失败	返回 "LLM error:..."，测试通过

单元测试

测试与演示：单元测试

模块名	测试对象	测试方法	测试结果
learning_plan	analyze_quiz_results	提供各种分数的测验结果	正确返回 (critical_areas, moderate_areas, good_areas) 列表，测试通过
		total_questions 为 0	避免除零错误，主题归入 critical_areas，测试通过
		所有分数大于 70	所有主题归入 good_areas，测试通过
	generate_plan	根据 analyze_quiz_results 生成计划	学习计划条目包含正确日期、优先级、主题及推荐材料，测试通过
		没有 quiz_results	返回空计划，测试通过
	recommend_materials	retriever 返回上下文	prompt 包含上下文，调用 LLM 返回材料列表，测试通过
		LLM 抛出异常	捕获异常并返回 ["No materials available"]，测试通过
	generate_plan_from_prompt	提供用户自定义目标列表	返回计划条目，每个目标优先级为 User-defined，日期间隔 2 天，测试通过
	display_plan	有学习计划条目	正确打印每个条目的日期、主题、优先级和材料，测试通过
		学习计划为空	只打印标题，无异常，测试通过
	save_to_file	正常保存计划	文件创建成功，包含 JSON 序列化的计划条目，日期格式为 ISO，测试通过
		指定不存在的 base_dir	自动创建目录，文件保存成功，测试通过
	边界条件	主题名称为空字符串	正常处理，计划条目主题为空，测试通过
	日志记录	recommend_materials 调用	确认 llm_logger.log_llm_call 被调用，测试通过

单元测试

测试与演示：单元测试

模块名	测试对象	测试方法	测试结果
database_creator	_process_node_recursive	输入完整节点数据，包含子节点和资源	在 session 中正确创建 CourseNode、Resource 节点及关系，测试通过
		节点缺少 name	不创建节点或关系，函数安全返回，测试通过
		节点资源为空或 None	不创建 Resource 节点，仍创建 CourseNode，测试通过
		存在多层嵌套子节点	所有子节点递归创建，关系正确，测试通过
	import_course_data	提供正确 JSON 文件	数据库清空，根节点创建成功，子节点及关系导入正确，返回 True，测试通过
		JSON 文件不存在	捕获 FileNotFoundError，打印错误，返回 False，测试通过
		JSON 格式错误	捕获 JSONDecodeError，打印错误，返回 False，测试通过
		JSON 缺少 root_name	打印错误信息，返回 False，测试通过
		JSON children 为空列表	仅创建根节点，无子节点或关系，返回 True，测试通过
		统计信息查询返回 None	打印“未能获取统计信息”，不报错，返回 True，测试通过
	visualize_graph	数据库返回多个节点和关系	HTML 文件生成成功，节点和边正确显示，调用 webbrowser.open 尝试打开，返回 True，测试通过
		保存 HTML 文件失败（权限或路径错误）	捕获异常并打印 traceback，返回 False，测试通过
		webbrowser.open 抛异常	捕获异常，打印提示手动打开文件，不影响 HTML 文件生成，返回 True，测试通过
	异常处理	Neo4j 驱动抛出异常	捕获异常，不导致程序崩溃，打印提示，测试通过

单元测试

模块名	测试对象	测试方法	测试结果
quiz_operations	prepare_quiz_questions	提供有效学科主题，retriever 返回上下文	返回非空问题列表，used_retriever=True，测试通过
		LLM 返回空主题列表	返回空列表，used_retriever 正确反映上下文使用情况，测试通过
		生成题目数多于 max_questions	每个主题生成不超过分配的题目数，总题目数 ≤ max_questions，测试通过
		topics 数量超过 max_topics	只保留前 5 个主题，测试通过
		题目格式不包含 “Correct Answer”	correct 字段为 "?"，测试通过
		batch 处理题目	每个主题正确生成题目列表，测试通过
	generate_quiz	questions 列表为空	打印警告 “No quiz topics generated.”，返回空字典，测试通过
		用户正确回答问题	user_scores 计数正确，输出显示正确分数，测试通过
		用户错误回答问题	user_scores 计数正确，输出显示正确答案，测试通过
		correct 为 "?" 的题目	自动计为正确答案，测试通过
		输入异常（如 input 抛异常）	捕获异常，不影响其他题目计分，测试通过
	generate_learning_plan_from_quiz	提供有效 quiz_results	调用 LearningPlan.generate_plan 返回学习计划，计划条目正确，测试通过
		quiz_results 为空	返回空学习计划，display_plan 输出标题，测试通过
		多主题 quiz_results	每个主题生成计划条目，优先级和日期间隔正确，测试通过
	异常处理	auto_answer 抛异常	generate_quiz 捕获异常，不影响其他题目计分，测试通过

单元测试

模块名	测试对象	测试方法	测试结果
summary_generator	generate_summary	输入普通主题文本，language="en"	返回生成的 summary 文本，used_retriever 为 True/False 取决于上下文是否存在，测试通过
		language="auto"，输入英文	调用 LanguageHandler.choose_or_detect 返回 "en"，生成 summary 正确，测试通过
		retriever 返回上下文	input_text 拼接上下文后生成 summary，used_retriever=True，测试通过
		retriever 返回空	summary 生成不依赖上下文，used_retriever=False，测试通过
		LLM 返回异常	捕获异常，确保函数抛出或安全处理（可通过 mock 测试），测试通过
		input_text 非常长	summary 生成成功，长度符合预期，测试通过
		input_text 为空字符串	返回空或最小 summary，不报错，测试通过

单元测试

测试与演示：单元测试

工具名	测试对象	测试方法	测试结果
auto_answer	looks_like_question	输入带问号的文本，如 "What is AI?"	返回 True，测试通过
		输入非问题文本，如 "This is a statement."	返回 False，测试通过
	auto_answer	输入问题文本并提供 mock AgentExecutor	调用 run_agent 并返回 True，测试通过
covert_resource	convert_to_pdf	输入存在的文档文件路径和有效输出目录	返回生成的 PDF 路径，文件存在，测试通过
		输入不存在的文件路径	捕获 FileNotFoundError 或返回 None，测试通过
		soffice 命令执行失败（模拟 subprocess.CalledProcessError）	返回 None，并记录错误日志，测试通过
edu_tools	wikipedia_search	输入存在的 Wikipedia 主题	返回非空摘要文本，测试通过
		输入不存在的 Wikipedia 主题	返回错误提示或异常信息，测试通过
	define_word	输入存在的单词	返回正确的定义字符串，测试通过
	calculator	输入有效数学表达式，如 "2+3*4"	返回计算结果 "14"，测试通过
	current_date / current_weekday	调用函数	返回当天日期和星期字符串，格式正确，测试通过
	detect_language	输入英文文本	返回 "en" 或对应语言代码，测试通过

单元测试

工具名	测试对象	测试方法	测试结果
ingest	load_file	提供不同类型文件（.txt, .pdf, .docx）	返回 Document 列表且 metadata 中包含正确的 source，测试通过
	ingest_folder	提供包含多个文档的文件夹	返回正确的文档数和分块数，调用 RAG 向量存储成功，测试通过
	main	模拟命令行参数调用 main	成功执行 ingest_folder 并处理指定目录，测试通过
language_handler	detect_language	提供中文、英文、未知语言文本	正确返回对应语言代码或默认“en”，测试通过
	set_language & get_language	设置语言为“zh”，再读取	返回“zh”，测试通过
	translate & ensure_language	将英文文本翻译为中文，确保语言一致	返回中文文本（或原文在失败情况下），测试通过
llm_logger	LLMLogger 单例实例	多次调用 get_llm_logger()，检查返回对象是否相同	返回同一实例，单例模式生效
	log_llm_call	模拟 LLM 响应对象并记录日志，检查 llm_log.json 内容是否增加并包含正确字段	日志成功写入文件，新增记录格式正确
	_extract_response_data	提供带或不带 response_metadata 的响应对象，检查返回字典结构是否正确	均返回结构化字典；metadata 存在时各字段正确映射，不存在时使用默认格式输出
ocr_service	OCRService.extract_text_from_image	使用已知文本的图像（bytes、文件路径、numpy数组）进行 OCR 提取，检查返回文本是否正确	返回的文本包含“TEST”，格式为多行字符串，OCR 提取正常，无异常抛出
	OCRService.__init__	初始化 OCRService 并检查 reader 对象是否创建成功	self.reader 为有效的 easyocr.Reader 实例，语言模型正常加载，无初始化错误
	get_ocr_service 单例	多次调用 get_ocr_service()，检查返回对象是否相同	返回的对象内存地址一致，仅创建一次实例，单例机制正常生效

单元测试

工具名	测试对象	测试方法	测试结果
pdf_ocr_loader	PDFOCRLoader.load	使用包含文本和扫描图片的 PDF 文件，检查返回的 Document 列表数量和内容是否正确	返回文档数量正确；文本页正常提取；扫描页触发 OCR 并成功生成文本；metadata 含 source 与 page 字段
	PDFOCRLoader._extract_text_from_image	使用已知文字的 PIL 图像，验证 OCR 提取文本是否正确	OCR 能正确识别主要文本内容；返回字符串非空；格式为按行拼接
	PDFOCRLoader.reader	初始化 loader 后访问 reader 属性，确保 easyocr.Reader 被正确创建	首次访问成功创建 Reader；再次访问未重复创建；languages 与 gpu 参数正确生效
rag_service	RAGService.get_retriever	初始化 RAGService，调用 get_retriever，检查返回对象类型是否为 Retriever 或 MultiQueryRetriever	返回对象类型正确，检索器参数完整，能正常调用其检索接口
	RAGService.ingest_paths	提供 PDF/Docx 文件或 Document 对象列表，检查文档是否被正确添加并去重，返回值是否为 None 或错误提示	文档全部成功写入，向量库中文档数量符合预期，重复内容未被重复插入
	RAGService._get_vectorstore	删除或破坏 persist_directory 后调用，验证是否能正确重建向量存储或使用内存存储	能成功初始化新向量库，未出现异常，后续插入与检索流程正常运行
rag_utils	get_context_or_empty 无 retriever	传入 query 且 retriever 为 None，检查返回值是否为空字符串	返回值为 ""，函数按预期退出
	get_context_or_empty 有 retriever 返回文档	使用 mock retriever 返回包含 page_content 的文档列表，检查返回值是否拼接正确	返回内容为文档内容以 \n\n 拼接，顺序正确
	get_context_or_empty retriever 抛异常	使用 mock retriever 抛出异常，检查函数是否返回空字符串且不崩溃	未抛出异常，返回 ""，日志记录被触发

单元测试

测试对象	测试方法	测试结果
StudySummaryGenerator + RAGService	调用 generate_summary 时使用 retriever 获取上下文	上下文正确注入 prompt，生成摘要包含检索信息
StudySummaryGenerator + LanguageHandler	设置 language="auto"	自动检测语言，生成摘要文本语言正确
QuizModule + get_context_or_empty	生成测验时获取相关文档	题目内容包含上下文信息
OCRService + PDFOCRLoader	提供扫描 PDF 或图片	OCR 提取文本成功，生成 Document 对象
PDFOCRLoader + RAGService	OCR 提取文本后嵌入向量库	文档可被检索，检索结果正确
LLMLogger + StudySummaryGenerator	调用 LLM 生成摘要	日志记录完整，包含 metadata、response
LLMLogger + QuizModule	调用 LLM 生成题目	日志记录每次 LLM 调用信息
LanguageHandler + auto_answer	输入多语言文本	系统检测语言并正确调用 LLM 返回答案
RAGService + ingest_paths	导入 PDF、Word、Markdown	文档正确加载、去重、嵌入向量库
RAGService + MultiQueryRetriever	使用 multi-query 检索	返回多版本问题，提高检索覆盖率

集成测试

测试与演示：系统测试

测试场景	测试方法	测试结果
用户登录接口可用性	使用正确与错误的用户名/密码、空字段、多次快速请求等情况调用 /login API	登录成功/失败逻辑正确；错误输入返回规范化错误提示；防暴力破解机制工作正常
用户登录状态保持	登录后访问多个受保护接口，检查 session/token 是否正确校验	Token 解析正确，未登录访问受阻，过期后强制刷新
用户信息加载	用户登录后加载个人基础信息（昵称、等级、学习记录）	数据准确，性能稳定，未发生延迟或丢失
实时测验内容生成接口	调用测验生成功能（调用 LLM）并检查生成内容质量、时延、重复度	题目生成稳定，无异常格式；响应延迟在可接受范围内
测验提交与评分	提交答案并检查评分、解析生成逻辑	批改结果与预期一致；解析段落结构完整；异常答案处理正常
测验历史记录写入	完成测验后检查数据库记录写入情况	全部记录写入正确，不重复、不缺失
学习进度计算	修改学习行为并重新查询学习进度接口	学习节点状态正确更新，无逻辑错误
可视化节点图接口	从知识图谱生成前端可视化节点数据，检查 JSON 输出结构	格式正确、节点关系清晰，无循环依赖错误
学习计划生成接口	输入用户模型数据，调用计划生成算法/LLM，检查输出稳定性	输出格式正确；学习节奏合理；异常输入处理完全
学习材料推荐	使用不同用户画像触发材料推荐模块	推荐结果与画像匹配度高；未出现空推荐
多终端兼容性（Web、Pad）	在不同浏览器及分辨率环境访问系统	UI 布局正常，无组件错位
并发访问压力测试	500–1000 并发用户访问测验生成接口	响应速度可控；无系统崩溃；自动扩容策略正常
数据库性能测试	频繁查询与写入学习记录、知识节点	查询速度稳定；无死锁；索引命中正常
权限系统测试	教师、学生、管理员分别访问各类接口	权限正常隔离；无越权访问
日志记录完整性测试	检查关键流程（测验生成/评分/计划生成）日志是否完整	日志完整、可追溯
文件上传	上传学习材料文件	文件类型校验正常

系统测试

测试与演示：验收测试

验收点	测试方法	预期结果
用户登录体验	使用真实用户账号进行全流程登录，登录后跳转	登录流程简洁；反馈明确；未出现无法进入系统情况
新用户首次登录引导	新用户首次登录后是否自动展示系统介绍、学习路线引导	引导内容准确，未出现空白页或卡顿
用户进行测验（实时生成测验内容）	真实用户进入“开始测验”，等待系统实时生成测验题目	题目生成速度可接受，题面清晰、难度匹配用户水平
用户提交测验并实时获得批改	用户提交答案后查看系统的自动批改结果、分数、解析	批改结果可信、反馈清晰
测验体验流畅度	用户连续完成多轮测验	无明显卡顿；无生成重复题目
用户查看学习进度（可视化节点图）	用户进入“学习进度”界面，查看知识图谱节点状态	图谱结构清晰；节点状态准确（已学/未学）
学习进度与实际学习行为一致性	学习多个内容后查看进度变化	图谱动态更新正常；无滞后或错误状态
制定学习计划（自动生成计划）	用户选择“制定计划”，系统生成个性化学习计划	计划内容合理、结构清晰；可直接执行
推荐学习材料	用户查看系统自动推荐的资料列表	推荐内容匹配用户水平与需求；未出现无关材料
教师查看学生学习进度	教师进入管理端查看学生的学习图谱	图谱渲染正确；数据与学生端一致
多角色切换与权限体验	用户从学生切换到教师帐号	无越权或错误菜单；权限边界清晰
系统整体体验	多名用户使用系统	无崩溃；主要流程顺畅；功能无重大缺陷
教师能成功上传文档并获得解析后的文本	教师上传 pdf 文件，查看前端是否解析成功	文档内容可见，无格式异常
学生能成功查看课件 pdf 和课程视频	选择多个 pdf 进行查看，选择多个视频长时间播放	文档显示正确，视频正常播放
用户提问可得到正确的多语言回答	用中文、英文、日文提问，检查系统是否语言一致、回答合理	输出语言与用户一致、语义正确
管理员可查看 LLM 调用日志或调试信息	执行多次询问后查看日志界面/文件	记录完整，数据结构清晰

验收测试

100%

测试通过！

现场演示

这里我们进行现场的演示，
按照要求，
已经搭建校内可访问的URL。
http://43.138.78.65:8000/



总结与展望

主要成果:

- ✅ 成功构建了完整的智能学习辅助系统
- ✅ 实现了多 Agent 协同的 HI+AI 工作模式
- ✅ 验证了 RAG 技术在教育场景的有效性
- ✅ 建立了完善的质量保障机制

完成度评估

完成度自评: 97%

- 核心功能全部实现
- 质量保障机制完善
- 前后端集成顺畅
- 待优化: 知识图谱可视化增强、多用户并发性能优化、AI 生成内容评价

97% 完成度

北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY

感谢各位的聆听

汇报人：刘奕名、牟龙雨、王麒宇、袁翊钧、蔡昊展

时　间：2026/10/22